TeachBook (Chemistry Sciences Template)

Faculty of Sciences

Álvaro Lozano Murciego and André Sales Mendes

Universidad de Salamanca

1st Edition • 2025 • ISBN: • DOI: • Ediciones Universidad de Salamanca

CONTENTS

I Tutorial 3

1 1. What is a TeachBook? 5
- 1.1 Attribution and origin . . . 6
- 1.2 What goes in and what comes out . . . 6
- 1.3 Why this is useful for teaching . . . 6
- 1.4 What this template adds on top of the original base . . . 6
- 1.5 What a TeachBook is NOT . . . 7
- 1.6 Practical advice to start . . . 8

2 Editing Workflow 9
- 2.1 Recommended Steps . . . 9
- 2.2 Tools . . . 9

3 Editing with AI 11
- 3.1 Prompts for the Assistant . . . 11
- 3.2 Prompt Example . . . 11

4 Web Publication 13
- 4.1 Steps to publish . . . 13

5 5. The Teaching-as-Code Philosophy 15
- 5.1 Code as a bridge to the final result . . . 15
- 5.2 What artifacts can be generated from code? . . . 16
- 5.3 Why this fits so well with AI . . . 16
- 5.4 Reproducibility, not just automation . . . 17
- 5.5 The mindset shift . . . 17

6 6. MyST Markdown: The Basics 19
- 6.1 Essential syntax . . . 19
- 6.2 Directives (special blocks) . . . 20
- 6.3 Code blocks . . . 21

7 7. Basic Notebooks for Teaching 23
- 7.1 Observations . . . 23

8 8. Interactive HTML in TeachBook 25
- 8.1 The `{raw} html` directive . . . 25
- 8.2 Example: styled table . . . 25
- 8.3 Example: collapsible content with `<details>` . . . 25

9 9. Live Code with Thebe (Optional) 27

9.1 How does it work? . . . 27
9.2 Syntax: the `{code-cell}` directive . . . 27
9.3 What do you need to enable it? . . . 27
9.4 Simpler alternatives . . . 28

II Examples by Degree **29**

10 Degree in Physics **31**
10.1 Physics Example: Harmonic Oscillator . . . 31
10.2 Drawing Circuits with SchemDraw . . . 32
10.3 Advanced CircuitikZ for Precise Circuits . . . 32
10.4 Technical Diagrams with Kroki for Physics . . . 34
10.5 Ohm's Law: Linear Fit of V-I Data . . . 37
10.6 Harmonic Oscillator: Simple Harmonic Motion . . . 38

11 Degree in Mathematics **41**
11.1 Mathematics Example: Symbolic Calculus . . . 41
11.2 Quadratic Equation with Manim . . . 41
11.3 Newton's Method for Finding Roots . . . 42
11.4 2D Linear Transformations . . . 43

12 Degree in Statistics **45**
12.1 Statistics Example: Data Generation . . . 45
12.2 Linear Regression with Synthetic Data . . . 45
12.3 Estimating π with Monte Carlo . . . 46
12.4 Practice Questions: Statistics . . . 47

13 Degree in Computer Science **51**
13.1 Entity-Relationship Diagram with Kroki (Mermaid) . . . 51
13.2 UML Class Diagram with Kroki (Mermaid) . . . 54
13.3 Diagrams with Kroki for Computer Science . . . 56
13.4 Algorithmic Complexity: Visual Comparison . . . 60

14 Degree in Geology **63**
14.1 Topographic Profile . . . 63
14.2 Stratigraphic Column . . . 64
14.3 The Rock Cycle with Kroki (Mermaid) . . . 65

III Cross-disciplinary Examples **67**

15 Cross-disciplinary Examples **69**
15.1 Example families . . . 69
15.2 The common idea . . . 70
15.3 Growth and Saturation Models . . . 70
15.4 Questionnaire Analysis . . . 71
15.5 Text Corpus Frequencies . . . 72
15.6 Rubrics and Learning Sequences . . . 73
15.7 Simulated Data and Variable Relationships . . . 75
15.8 Balances and Conservation . . . 76

IV Basic Content 79

16 Basic Content 81
16.1 What you will find here . . . 81
16.2 How to use this section . . . 81

17 Figures and Images 83
17.1 Centered image with alt text . . . 83
17.2 Numbered figure with caption . . . 83
17.3 Cross-reference to a figure . . . 84
17.4 Margin figure . . . 84
17.5 Useful parameters . . . 85

18 Equations and Mathematics 87
18.1 Inline math . . . 87
18.2 Display equation (unnumbered) . . . 87
18.3 Numbered equation with label . . . 87
18.4 Cross-reference to an equation . . . 88
18.5 Common math symbols . . . 88

19 Citations and Bibliography 89
19.1 References file . . . 89
19.2 Textual citation: `{cite:t}` . . . 89
19.3 Parenthetical citation: `{cite:p}` . . . 90
19.4 Multiple citations . . . 90
19.5 Printing the bibliography . . . 90
19.6 Workflow . . . 90

20 Cross-References 91
20.1 Label a section . . . 91
20.2 Reference a section . . . 91
20.3 Label and reference figures . . . 91
20.4 Label and reference equations . . . 92
20.5 Label and reference tables . . . 92
20.6 Roles summary . . . 93

21 Tables 95
21.1 Simple Markdown table . . . 95
21.2 Numbered table with caption . . . 95
21.3 List-table directive . . . 96
21.4 Tips for readable tables . . . 97

22 Admonitions and Dropdowns 99
22.1 Note (`{note}`) . . . 99
22.2 Warning (`{warning}`) . . . 99
22.3 Tip (`{tip}`) . . . 99
22.4 Important (`{important}`) . . . 100
22.5 Caution (`{caution}`) . . . 100
22.6 Custom admonition . . . 100
22.7 Dropdown (collapsible content) . . . 100
22.8 Teaching use cases . . . 101

23 Tabs and Cards 103
23.1 Simple tabs (`{tabbed}`) . . . 103
23.2 Tabs for procedure steps . . . 104

23.3 Cards (`{card}`) . . . 105

24 Code Blocks **107**
24.1 Syntax highlighting . . . 107
24.2 Line numbers . . . 107
24.3 Highlight specific lines . . . 108
24.4 Other languages . . . 108
24.5 Common languages in science . . . 109

25 Videos **111**
25.1 YouTube video (HTML + PDF) . . . 111
25.2 How to get a YouTube video ID . . . 111
25.3 Local video (HTML5) . . . 112
25.4 Animation generated with Manim Community . . . 112
25.5 Video with description . . . 113
25.6 Recommended complete pattern . . . 113

26 Diagrams with Kroki (Mermaid) **115**
26.1 Flowchart (`flowchart`) . . . 115
26.2 Sequence diagram (`sequenceDiagram`) . . . 115
26.3 Class diagram (`classDiagram`) . . . 116
26.4 Entity-relationship diagram (`erDiagram`) . . . 118
26.5 State diagram (`stateDiagram-v2`) . . . 120
26.6 Flowchart directions . . . 121

27 HTML and PDF Compatibility **123**
27.1 Compatibility table . . . 123
27.2 Universal pattern: HTML + LaTeX fallback . . . 123
27.3 Example: video with fallback . . . 124
27.4 Example: diagram with Kroki . . . 124
27.5 Practical rules . . . 124

V Interactivity and Assessment **125**

28 Interactivity and Assessment **127**
28.1 What will you find here? . . . 127
28.2 Thebe + Binder: Live Code . . . 128
28.3 Interactive Notebooks . . . 130
28.4 Interactive Widgets (ipywidgets) . . . 131
28.5 Quizzes with jupyterquiz . . . 132
28.6 Parametric Questions . . . 133
28.7 Exercises with Solutions . . . 134
28.8 Interactive HTML (no frameworks) . . . 136
28.9 H5P (Optional) . . . 136

VI Information **139**

29 About **141**
29.1 How this book is made . . . 141
29.2 How to reproduce the website . . . 141
29.3 About the authors . . . 142

30 Licenses **143**

30.1 Book license . . . 143
30.2 External resources . . . 143
30.3 Code and tools . . . 143

31 How to Cite **145**
31.1 Text Citation . . . 145
31.2 BibTeX . . . 145
31.3 DOI . . . 145

32 References **147**

Bibliography **149**

Welcome to the **TeachBook Sciences Template**.

What is this?

This is a template designed for the faculty of the **Faculty of Sciences at USAL** to create interactive teaching books easily.

Content

In this book you will find:

- *Tutorials* to learn how to use the template
- *Examples by Degree* to see real cases
- Information on *how to cite* and *licenses*

PDF version

You can also download the printable version of the book:

- Download PDF in English
- Descargar PDF en español

> **Note**
>
> This project is designed to be used with **VS Code** and **AI** assistants.

Part I

Tutorial

CHAPTER

ONE

1. WHAT IS A TEACHBOOK?

A **TeachBook** is a digital teaching book built from text files, code, and multimedia resources, which are then compiled into a **navigable website** and, in many cases, also a **PDF version**.

In simple terms:

- you edit the **source content**
- the system **builds** it
- you get a **clean, functional, publishable** book

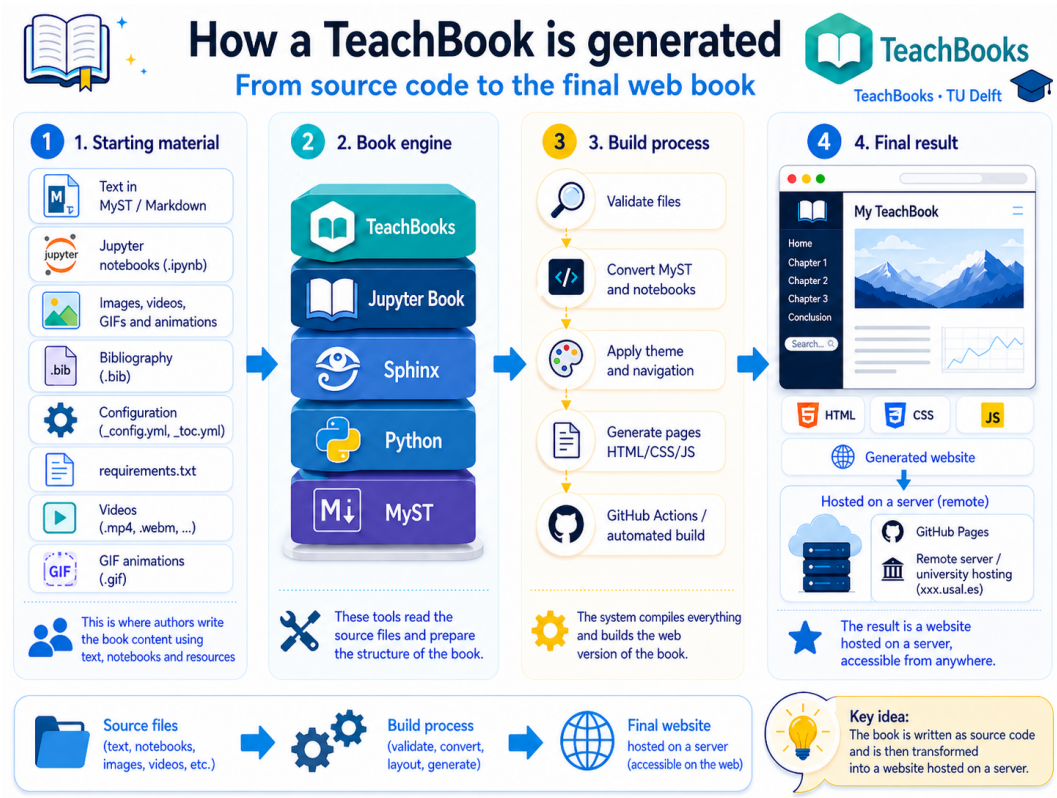


Fig. 1.1: A TeachBook starts from editable content and produces a publishable teaching website.

1.1 Attribution and origin

This template is built on the original ecosystem of:

- TeachBooks
- Jupyter Book

This specific adaptation has been prepared for the **Faculty of Sciences at the University of Salamanca (USAL)**.

This matters: **we are not starting from scratch**. We are starting from existing, tested, maintained tools, and building a teaching-focused template on top of them.

1.2 What goes in and what comes out

The core idea is this:

The following table summarizes the main elements of this section.

Table. What goes in and what comes out.

You start with...	And you get...
`.md` and `.ipynb` files	book web pages
images, bibliography, and static assets	enriched content
book configuration (`_config_*.yml`)	navigation, theme, language, extensions
table of contents (`_toc_*.yml`)	visible book structure

1.3 Why this is useful for teaching

1. **Accessibility**: the book can be read on the web from any device.
2. **Reproducibility**: the content comes from source files that can be versioned.
3. **Maintainability**: updating a page is easier than rebuilding a whole set of notes.
4. **Scalability**: you can start with one page and grow gradually.
5. **Compatibility**: the same project can produce both web and PDF outputs.

1.4 What this template adds on top of the original base

On top of TeachBooks / Jupyter Book, this project adds several practical improvements for teachers and students:

1.4.1 1. Multilingual structure

The book is prepared to work in:

- Spanish
- English

with synchronized content and table of contents structure.

1.4.2 2. PDF export

Besides the website, this template also supports a PDF output for printing or offline distribution.

1.4.3 3. Simple automation scripts

You do not need a complicated workflow on day one. The project includes scripts to:

- prepare the environment
- build the website
- open live preview
- export PDF
- convert PDF to Markdown

1.4.4 4. AI agent integration

The template is prepared so an agent can help you:

- create content
- reorganize chapters
- add multimedia
- maintain the structure of the book

1.5 What a TeachBook is NOT

It is not:

- a PowerPoint presentation
- a React-style web app with heavy JS frameworks
- a static PDF pretending to be a website

The main idea is that the content remains **editable, structured, and reusable**.

1.6 Practical advice to start

Do not try to build a huge book on the first day.

Start small:

- a cover page
- one short chapter
- one image
- one equation
- one section with two or three pages

Once that works well, expand.

CHAPTER

TWO

EDITING WORKFLOW

The workflow with **TeachBook** is designed to be simple and efficient.

2.1 Recommended Steps

1. **Write content**: Use Markdown or Notebooks (`.ipynb`).
2. **Preview**: Run `python scripts/launch_preview.py` to see the same build that will be published.
3. **Commit & Push**: Use the Git extension in VS Code to save your work.
4. **Deploy**: GitHub Actions will automatically publish the updated version.

2.2 Tools

- **VS Code**: Main editor.
- **Jupyter Extension**: For editing notebooks interactively.
- **GitHub**: For version control and hosting.
- `_toc.yml`: Table of Contents.
- `_config.yml`: General configuration.

CHAPTER

THREE

EDITING WITH AI

This template is optimized for use with Artificial Intelligence assistants (like GitHub Copilot, ChatGPT, or Claude).

3.1 Prompts for the Assistant

You can ask the assistant to:

- "Create a new chapter about Thermodynamics."
- "Add a matplotlib graph to visualize this function."
- "Correct the style of this paragraph."

The assistant knows the project structure thanks to the `Agent.md` file.

3.2 Prompt Example

> "Create a code cell that plots the standard normal distribution using matplotlib."

The AI will give you the code and you just have to run it or include it in your book.

CHAPTER

FOUR

WEB PUBLICATION

Publication is done automatically through **GitHub Pages**.

4.1 Steps to publish

1. Ensure your repository is public (or you have access to GitHub Pages in private).
2. Make changes to your content.
3. Save changes with a **Commit** and **Push** to the main branch (`main`).
4. A "GitHub Action" will run automatically and build your book.
5. In a few minutes, you will see your updated book at your repository URL (configured in Settings > Pages).

CHAPTER

FIVE

5. THE TEACHING-AS-CODE PHILOSOPHY

The central idea is not "learning to program" in the abstract. The idea is to use programming languages and markup languages as **intermediate languages** for generating teaching artifacts: images, diagrams, animations, videos, simulations, quizzes, tables, notebooks, and PDFs.

In other words: working with **X as code**.

- *Diagrams as code*: diagrams written as text.
- *Figures as code*: plots and figures generated from data or formulas.
- *Animations as code*: educational videos rendered from a programmed scene.
- *Quizzes as code*: questions, solutions, and exercise banks defined as structured text.
- *Books as code*: a complete book compiled from Markdown files, notebooks, and configuration.

Key idea

A teaching artifact generated from code is not edited "by hand" at the end of the process. It is described with an intermediate language, versioned, and regenerated as many times as needed.

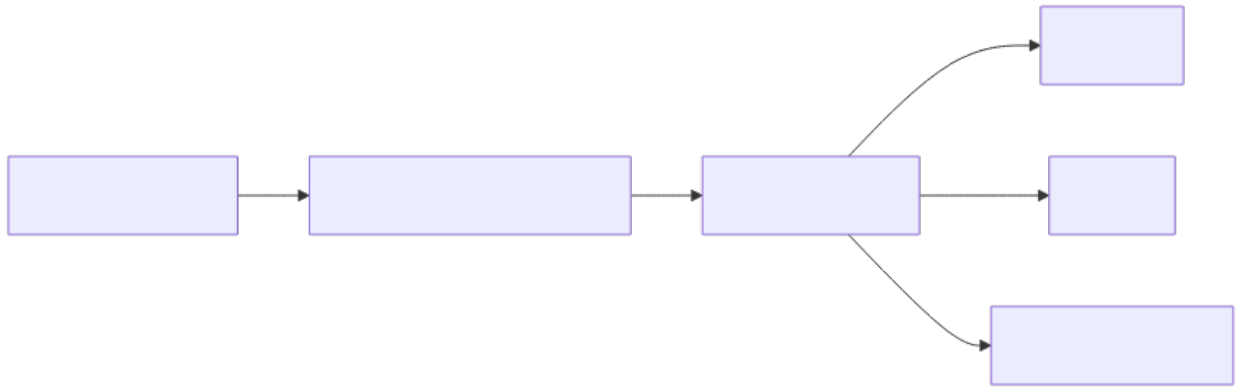
5.1 Code as a bridge to the final result

When you draw a diagram directly in a visual tool, the result is locked inside that interface. If you want to change ten arrows, translate labels, or adapt the style, you have to edit it manually again.

When you describe that same diagram with code, an intermediate layer appears:

The following diagram visually summarizes this part of the explanation.

Diagram: Code as a bridge to the final result.



That intermediate layer is where AI works especially well. An assistant can read, modify, translate, refactor, and compare structured text much better than it can directly manipulate a PDF, a final image, or a closed presentation.

5.2 What artifacts can be generated from code?

The following table summarizes the main elements of this section.

Table. What artifacts can be generated from code?.

Final artifact	Intermediate language	Teaching use case
Diagram	Mermaid/Kroki, GraphViz, PlantUML	Concept map, experiment flow, process schema
Scientific plot	Python + matplotlib	Represent real or simulated data
Electrical circuit	Schemdraw or CircuitikZ	Generate consistent schematics for Physics problems
Animation or video	Manim Community	Explain a transformation, a wave, or a proof step by step
Table or dataset	Python + pandas	Create clean tables from raw data
Quiz	MyST, JupyterQuiz, structured Markdown	Review questions with collapsible solutions
Executable notebook	Jupyter Notebook	Simulation, reproducible calculation, guided practice
Web book and PDF	Jupyter Book/TeachBooks	Publish the same material as a website, download, and printable version

5.3 Why this fits so well with AI

AI does not "understand" a final image the way it understands a code-based description. If you give it a screenshot of a diagram, it can suggest changes; if you give it the Mermaid or Kroki code that generates it, it can **make** those changes.

The practical advantage

We do not ask AI to edit pixels. We ask it to edit instructions. Then the system recompiles those instructions and produces the final artifact.

This makes it possible to ask for things like:

- "Turn this flowchart into a state diagram."
- "Generate an English version while keeping the same structure."
- "Adapt the example so it uses Biology data instead of Physics data."
- "Create a 20-second animation showing this function growing."
- "Create three versions of the quiz with different difficulty levels."

The key is that the final result is not handcrafted every time. It is produced from an editable recipe.

5.4 Reproducibility, not just automation

Working "as code" does not mean making things more complicated. It means every artifact has a clear recipe:

1. **Input**: data, formulas, text, or a teaching idea.
2. **Intermediate language**: Markdown, Python, Mermaid, LaTeX, Manim, etc.
3. **Compilation**: the system transforms that description.
4. **Output**: image, video, website, PDF, notebook, or quiz.

If you change the recipe, you regenerate the artifact. If something fails, you fix the recipe. If another teacher wants to adapt it, they do not start from scratch: they modify the recipe.

5.5 The mindset shift

The question stops being: "Which program should I use to draw this?"

The question becomes: "Which intermediate language best describes this artifact?"

Not every material needs code. But when a resource will be revised, translated, versioned, reused, or generated with AI support, it is worth asking whether it can be expressed as **X as code**.

That is the teaching-as-code philosophy: using code not as an end in itself, but as a **generative medium** between the teaching intention and the final artifact.

CHAPTER

SIX

6. MYST MARKDOWN: THE BASICS

MyST (Mark up Your Scientific Text) is a Markdown extension designed for scientific and technical documents. It is the format used by Jupyter Book. If you know basic Markdown, you already know 80 % of MyST.

6.1 Essential syntax

6.1.1 Headings

```
# Main title (H1)
## Section (H2)
### Subsection (H3)
```

6.1.2 Text formatting

```
**Bold** for strong emphasis.
*Italic* for terms or titles of works.
`inline code` for file names or commands.
```

6.1.3 Lists

```
- Unordered list item
- Another item
  - Sub-item with indentation

1. First step
2. Second step
3. Third step
```

6.1.4 Links and images

```
[Link text](https://example.com)
```

```
![Alt text](path/to/image.png)
```

6.2 Directives (special blocks)

MyST adds **directives** that don't exist in standard Markdown:

Note

This is a note. It appears highlighted in blue.

Warning

This is a warning. It appears highlighted in red/orange.

Tip

This is a tip. It appears highlighted in green.

You can create custom blocks with `admonition`:

```
```{admonition} My custom title
Block content goes here.
```
```

```
## Math

Inline: the equation $E = mc^2$ is written as `$E = mc^2$`.

Display:

$$
\int_0^\infty e^{-x^2} \, dx = \frac{\sqrt{\pi}}{2}
$$

Written as `$$ ... $$` on its own line.

## Tables

```md

The following table summarizes the main elements of this section.

**Table. Tables.**
```

(continues on next page)
```

(continued from previous page)

```
| Column A | Column B | Column C |
|----------|----------|----------|
| data 1   | data 2   | data 3   |
| data 4   | data 5   | data 6   |
```

Result:

The following table summarizes the main elements of this section.

Table. Tables.

Column A	Column B	Column C
data 1	data 2	data 3
data 4	data 5	data 6

6.3 Code blocks

```python
def greet(name):
    return f"Hello, {name}!"
```

Tip

Don't try to memorize all the syntax. Use an AI assistant to generate the MyST you need and learn as you go.

CHAPTER

SEVEN

7. BASIC NOTEBOOKS FOR TEACHING

A **Jupyter Notebook** is a document that mixes explanatory text and executable code in cells. It is ideal for teaching: you can explain a concept and show the calculation in the same document.

Below is a simple example: computing and plotting the function $y = x^2$.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 100)
y = x ** 2

print(f"Minimum value of y: {y.min()}")
print(f"Value at x=3: {3**2}")
```

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x, y, label=r"$y = x^2$", color="#2563eb", linewidth=2)
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Quadratic function")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

7.1 Observations

- The function $y = x^2$ is a parabola with its vertex at the origin.
- Change the exponent in `y = x ** 2` to `x ** 3` and see how the plot changes.
- Notebooks let you experiment directly: modify the values and re-run the cell.

CHAPTER

EIGHT

8. INTERACTIVE HTML IN TEACHBOOK

You can embed HTML directly into your TeachBook pages using the `{raw} html` directive. This lets you create interactive content **without needing JavaScript frameworks** like React or Vue.

8.1 The `{raw} html` directive

Write the following in your `.md` file:

```
```{raw} html
<your HTML here>
```
```

Everything inside is inserted as-is into the web page.

8.2 Example: styled table

8.3 Example: collapsible content with `<details>`

This is perfect for hiding exercise solutions so students can reveal them when they are ready.

Warning

Content inserted with `{raw} html` **does not appear in the PDF export**. If you need content in the PDF, use standard MyST syntax (Markdown tables, `admonition` directives, etc.) instead.

Tip

Combine HTML for interactive elements (collapsibles, styling) with standard MyST for main content. This way your book works well in both web and PDF.

CHAPTER

NINE

9. LIVE CODE WITH THEBE (OPTIONAL)

Thebe is a technology that lets you run Python code **directly in the browser** from your web book. Students can modify the code and see results without installing anything.

> **Warning**
>
> This feature is **optional** and requires additional configuration of a Jupyter server or Binder. You do not need it for the basic TeachBook setup.

9.1 How does it work?

Thebe connects your web book to a Jupyter server in the cloud. When the reader clicks a "Run" button, the code executes on that server and the result appears on the page.

9.2 Syntax: the `{code-cell}` directive

Instead of a normal code block, use the `{code-cell}` directive:

```
```{code-cell} python
import numpy as np
print(np.sqrt(16))
```
```

This turns the code block into an executable cell if Thebe is configured.

9.3 What do you need to enable it?

1. An accessible Jupyter server (can be free Binder).
2. Configure Thebe options in your `_config.yml`.
3. Enable execution buttons in the interface.

Note

If you only need to display static code with syntax highlighting, use normal code blocks (```python). Thebe is only necessary if you want readers to **execute** the code.

9.4 Simpler alternatives

If Thebe seems too complex, consider these options:

- **`.ipynb` notebooks**: Executed when the book is built (if you enable `execute_notebooks: auto`).
- **Interactive HTML**: Use `<details>` and CSS for simple interactions (see previous chapter).
- **Google Colab links**: Readers can open notebooks in the cloud with one click.

Summary

Thebe is powerful but optional. Start with static content and notebooks, and consider Thebe when your students need to experiment with code in real time.

Part II

Examples by Degree

CHAPTER

TEN

DEGREE IN PHYSICS

In this section, you will find examples adapted to Physics Degree courses. The goal is to show how to integrate:

- Complex mathematical formulas
- Plots of physical simulations
- Experimental data analysis code

10.1 Physics Example: Harmonic Oscillator

This notebook simulates a simple harmonic oscillator.

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
k = 1.0  # Spring constant
m = 1.0  # Mass
omega = np.sqrt(k / m)
t = np.linspace(0, 20, 100)

# Position
x = np.cos(omega * t)

# Plot
plt.figure(figsize=(8, 4))
plt.plot(t, x)
plt.title('Simple Harmonic Oscillator')
plt.xlabel('Time (s)')
plt.ylabel('Position (m)')
plt.grid(True)
plt.show()
```

10.2 Drawing Circuits with SchemDraw

In this example we draw a series RC circuit (battery + resistor + capacitor) using the `schemdraw` library. This type of circuit is fundamental in electronics and appears in low-pass filters, timers, and coupling circuits.

```
import schemdraw
import schemdraw.elements as elm

with schemdraw.Drawing(file='rc_circuit.png') as d:
    d += elm.Battery().up().label('V', loc='bot')
    d += elm.Resistor().right().label('R')
    d += elm.Capacitor().right().label('C')
    d += elm.Line().down()
    d += elm.Line().tox(d._elements[0].start).left()

d.draw()
```

10.2.1 Circuit Components

- **Battery (V):** provides the electromotive force that drives the current.
- **Resistor (R):** limits current flow and dissipates energy as heat.
- **Capacitor (C):** stores energy as an electric field between its plates.

The time constant of the circuit is $\tau = RC$, which determines the charging and discharging speed of the capacitor.

```
R = 1000     # 1 kΩ
C = 1e-6     # 1 μF
tau = R * C
print(f'Time constant τ = RC = {tau*1e3:.2f} ms')
print(f'Time for full charge (~5τ) = {5*tau*1e3:.2f} ms')
```

10.2.2 Conclusion

With `schemdraw` we can generate clean, professional circuit diagrams. The time constant $\tau = RC$ is the key parameter: after a time 5τ the capacitor is considered fully charged (≈99.3% of the source voltage).

10.3 Advanced CircuitikZ for Precise Circuits

CircuitikZ is the advanced option for drawing circuits with LaTeX-quality output. If you want highly precise schematics, similar to technical notes or scientific papers, this is the right tool.

Practical recommendation

Use **SchemDraw** for most teaching examples. Use **CircuitikZ** when you need a more formal and precise finish.

10.3.1 Recommended workflow in TeachBook

In this project, CircuitikZ is not inserted directly as `raw latex` in the page. Instead:

1. You write the circuit in a `.tex` file
2. You convert it to an image with a script
3. You insert the image with `{figure}`

This makes it work in both HTML and PDF.

10.3.2 Example source file

File `rc_circuit.tex`:

```
\begin{circuitikz}
\draw
  (0,0) to[battery1,l=$V$] (0,3)
  to[R,l=$R$] (3,3)
  to[C,l=$C$] (3,0)
  -- (0,0);
\end{circuitikz}
```

10.3.3 Command to render it

```
python scripts/render_circuitikz.py rc_circuit.tex book/_static/generated/rc_circuit.
↪png
```

10.3.4 Insert it in a MyST page

```
```{figure} ../../../_static/generated/rc_circuit.png
:alt: RC circuit generated with CircuitikZ
:width: 70%
:align: center
:name: fig-advanced-circuitikz-1

RC circuit generated with CircuitikZ.
```
```

```
## Rendered result


The {numref}`fig-advanced-circuitikz-2` visually summarizes this part of the␣
↪explanation.

```{figure} ../../../_static/generated/rc_circuit_circuitikz.png
:alt: RC circuit generated with CircuitikZ
:width: 70%
:align: center
:name: fig-advanced-circuitikz-2

RC circuit generated from a `.tex` file with CircuitikZ and converted to a PNG image.
```
```

The advantage of this workflow is that the final circuit behaves like a normal image: it appears on the website, is included in the PDF, and does not require the reader to have LaTeX installed.

10.3.5 When to use CircuitikZ

- formal electrical schematics
- advanced electronics notes
- material with a technical-paper style finish

10.3.6 When to use SchemDraw instead

- quick classroom examples
- introductory material
- situations where you want to edit the circuit with less friction

10.3.7 Prerequisite

If you do not have Tectonic installed yet:

```
python scripts/setup_latex.py --yes
```

10.4 Technical Diagrams with Kroki for Physics

Kroki is useful for **block diagrams, signal flow, timing diagrams, and conceptual schematics**. For **electrical circuits with standard symbols**, the recommended tool is **CircuitikZ rendered as an image**.

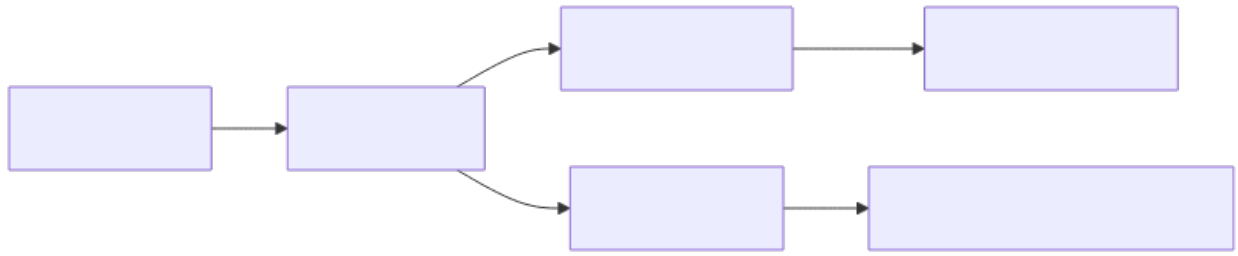
10.4.1 When to use each tool

- **Use Kroki** for conceptual explanations, functional blocks, timing, instrumentation, and signal flow.
- **Use WaveDrom through Kroki** for digital signals, buses, clocks, and timing protocols.
- **Use CircuitikZ** for resistors, capacitors, sources, switches, and formal electrical schematics.
- **Use SchemDraw** when you want to build simple circuits from Python inside a notebook.

10.4.2 1. Mermaid: instrumentation chain

The following diagram visually summarizes this part of the explanation.

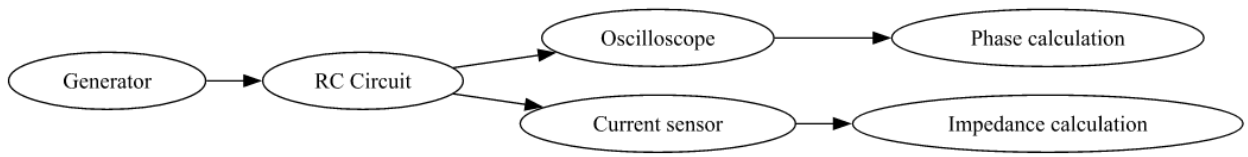
Diagram: Mermaid: instrumentation chain.



10.4.3 2. GraphViz: signal flow in an experimental setup

The following diagram visually summarizes this part of the explanation.

Diagram: GraphViz: signal flow in an experimental setup.



10.4.4 3. CircuitikZ: teaching circuit as an image

The following circuit is not drawn by hand: it comes from a `.tex` file with CircuitikZ code, rendered beforehand as an image so it works the same way in HTML and PDF.

The Fig. 10.1 visually summarizes this part of the explanation.

```
\begin{circuitikz}
\draw
  (0,0) to[V, l=$5\,\mathrm{V}$] (0,3)
  to[R, l=$R_1$] (3,3)
  to[C, l=$20\,\mu\mathrm{F}$] (3,0)
  -- (0,0);
\end{circuitikz}
```

Important

Kroki supports TikZ, but the public Kroki service does not guarantee that the `circuitikz` package is available. For robust teaching circuits, this project renders CircuitikZ locally as an image.

10.4.5 4. Wavedrom: input and output signals

The following diagram visually summarizes this part of the explanation.

Diagram: Wavedrom: input and output signals.

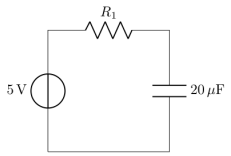
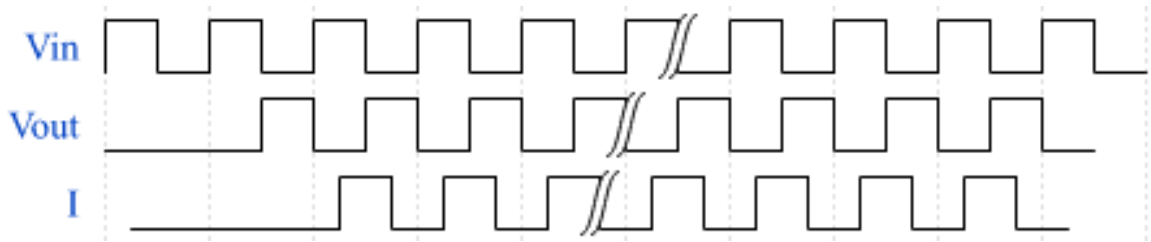


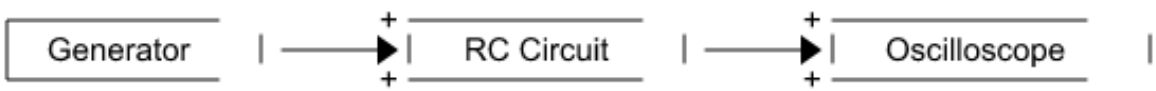
Fig. 10.1: RC circuit generated from CircuitikZ code.



10.4.6 5. Ditaa: quick experimental bench sketch

The following diagram visually summarizes this part of the explanation.

Diagram: Ditaa: quick experimental bench sketch.



10.4.7 Key idea

Kroki, CircuitikZ, and SchemDraw **complement** each other. They do not compete:

- **Kroki** = system-level visual explanation, blocks, flows, and timing diagrams.
- **CircuitikZ** = formal circuit with standard symbols and LaTeX-quality output.
- **SchemDraw** = programmable circuit from Python, ideal for teaching notebooks.

10.5 Ohm's Law: Linear Fit of V-I Data

We generate synthetic voltage-current data with noise and fit a linear regression $V = IR$ to verify Ohm's Law. We calculate the R^2 coefficient to evaluate the quality of the fit.

```
import numpy as np
import matplotlib.pyplot as plt

R_true = 220.0  # True resistance in Ω
np.random.seed(42)

I = np.linspace(0.001, 0.05, 30)
V_true = R_true * I
noise = np.random.normal(0, 0.3, size=V_true.shape)
V_measured = V_true + noise

coef = np.polyfit(I, V_measured, 1)
R_fitted = coef[0]
V_fit = np.polyval(coef, I)

SS_res = np.sum((V_measured - V_fit) ** 2)
```

(continues on next page)

(continued from previous page)

```
SS_tot = np.sum((V_measured - np.mean(V_measured)) ** 2)
R2 = 1 - SS_res / SS_tot

print(f'True resistance:     {R_true:.1f} Ω')
print(f'Fitted resistance:   {R_fitted:.1f} Ω')
print(f'R² = {R2:.6f}')
```

10.5.1 Visualization

The plot shows experimental data (dots) and the fitted line. An R^2 close to 1 indicates that the linear model explains the variability in the data.

```
plt.figure(figsize=(8, 5))
plt.scatter(I * 1000, V_measured, color='steelblue', label='Measured data', zorder=3)
plt.plot(I * 1000, V_fit, 'r-', linewidth=2, label=f'Fit: V = {R_fitted:.1f}·I')
plt.xlabel('Current I (mA)')
plt.ylabel('Voltage V (V)')
plt.title(f"Ohm's Law — Linear fit  (R² = {R2:.4f})")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

10.5.2 Conclusion

The linear fit recovers the resistance value with great accuracy. The $R^2 \approx 1$ coefficient confirms that the $V = IR$ relationship is linear, validating Ohm's Law for this synthetic dataset.

10.6 Harmonic Oscillator: Simple Harmonic Motion

Simple harmonic motion describes systems that oscillate periodically:

$x(t) = A\cos(\omega t + \varphi)$

where A is the amplitude, ω the angular frequency, and φ the initial phase. By modifying these parameters we can explore different oscillatory behaviors.

```
import numpy as np
import matplotlib.pyplot as plt

A = 2.0          # Amplitude (m)
omega = 2*np.pi  # Angular frequency (rad/s) → 1 Hz
phi = np.pi/4    # Initial phase (rad)

t = np.linspace(0, 4, 1000)
x = A * np.cos(omega * t + phi)

plt.figure(figsize=(10, 4))
plt.plot(t, x, linewidth=2, color='darkorange')
plt.axhline(0, color='gray', linewidth=0.5)
plt.xlabel('Time (s)')
```

(continues on next page)

(continued from previous page)

```
plt.ylabel('Displacement x (m)')
plt.title(f'Harmonic Oscillator  A={A}, ω={omega:.2f}, φ={phi:.2f}')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

10.6.1 Effect of Parameters

You can experiment by changing `A`, `omega`, and `phi` in the cell above:

- **Amplitude (A):** controls the maximum height of the oscillation.
- **Angular frequency (ω):** controls the oscillation speed ($T = 2\pi/\omega$).
- **Initial phase (φ):** shifts the curve horizontally.

```
A = 2.0
phi = 0
t = np.linspace(0, 4, 500)

fig, axes = plt.subplots(1, 3, figsize=(14, 3.5), sharey=True)

for ax, omega_val, color in zip(axes, [np.pi, 2*np.pi, 4*np.pi],
                                ['steelblue', 'darkorange', 'crimson']):
    x = A * np.cos(omega_val * t + phi)
    ax.plot(t, x, linewidth=2, color=color)
    ax.set_title(f'ω = {omega_val/np.pi:.0f}π rad/s')
    ax.set_xlabel('Time (s)')
    ax.grid(True, alpha=0.3)

axes[0].set_ylabel('x (m)')
plt.suptitle('Frequency comparison', y=1.02)
plt.tight_layout()
plt.show()
```

10.6.2 Conclusion

The simple harmonic oscillator is the basic model for oscillatory phenomena. Real systems such as pendulums (for small angles), springs, and LC circuits follow this behavior. The angular frequency $\omega = \sqrt{k/m}$ connects the motion to the physical properties of the system.

CHAPTER

ELEVEN

DEGREE IN MATHEMATICS

Examples for mathematics courses. Priorities here include:

- Rigor in mathematical notation (LaTeX)
- Visual proofs
- Symbolic algorithms (SymPy)

11.1 Mathematics Example: Symbolic Calculus

We will use SymPy to differentiate and integrate functions.

```
from sympy import symbols, diff, integrate, sin, exp

x = symbols('x')
f = exp(-x) * sin(x)

# Derivative
df = diff(f, x)
print(f"Derivative: {df}")

# Indefinite Integral
int_f = integrate(f, x)
print(f"Integral: {int_f}")
```

11.2 Quadratic Equation with Manim

In this section, we will demonstrate how to embed an educational animation generated through Python code using **Manim Community Edition**. The video explains the solutions to the quadratic equation depending on its discriminant Δ.

Video: The Quadratic Equation. Check the HTML version of the book to play the animation generated with Manim.

11.2.1 Manim Code

The video above was generated by running a Python script with Manim, where each animation is controlled step-by-step. It is a fantastic tool to create rigorous and visual educational material.

The recommended source file for this animation is located at:

- `manim_videos/2026-04-25_ecuacion_segundo_grado/scene.py`

11.3 Newton's Method for Finding Roots

The Newton-Raphson method is an iterative algorithm for finding roots of $f(x) = 0$:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

We apply it to the equation $x^3 - 2x - 5 = 0$, which has a real root near $x = 2$.

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x**3 - 2*x - 5

def df(x):
    return 3*x**2 - 2

x0 = 2.0
steps = [x0]
for _ in range(5):
    x_new = steps[-1] - f(steps[-1]) / df(steps[-1])
    steps.append(x_new)

for i, x in enumerate(steps):
    print(f'x_{i} = {x:.10f}   f(x) = {f(x):.2e}')
```

11.3.1 Convergence Visualization

The plot shows the function $f(x)$ and the tangent lines used at each iteration. Each tangent intersects the x-axis at the next approximation of the root.

```
x_plot = np.linspace(0.5, 3.5, 300)

plt.figure(figsize=(8, 5))
plt.plot(x_plot, f(x_plot), 'b-', linewidth=2, label=r'$f(x) = x^3 - 2x - 5$')
plt.axhline(0, color='gray', linewidth=0.8)

colors = ['red', 'orange', 'green', 'purple']
for i in range(min(4, len(steps)-1)):
    xi = steps[i]
    tangent = f(xi) + df(xi) * (x_plot - xi)
    plt.plot(x_plot, tangent, '--', color=colors[i], alpha=0.6,
             label=f'Iteration {i+1}')
    plt.plot(xi, f(xi), 'o', color=colors[i], markersize=6)
```

(continues on next page)

(continued from previous page)

```
plt.plot(steps[-1], 0, 'k*', markersize=12, label=f'Root ≈ {steps[-1]:.6f}')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title("Newton's Method — Convergence")
plt.legend(fontsize=8)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

11.3.2 Conclusion

Newton's method converges very quickly (quadratic convergence): in just 4-5 iterations we reach a precision of 10^{-10}. A good starting point is required, and $f'(x_n) \neq 0$ must hold at each step.

11.4 2D Linear Transformations

A linear transformation in $\mathbb{R}^2$ is represented by a 2×2 matrix. Applied to the unit square, it lets us visualize how the transformation distorts space. We explore three classic examples: rotation, scaling, and shear.

```
import numpy as np
import matplotlib.pyplot as plt

square = np.array([[0,0],[1,0],[1,1],[0,1],[0,0]]).T

theta = np.pi / 4
R_rot = np.array([[np.cos(theta), -np.sin(theta)],
                  [np.sin(theta),  np.cos(theta)]])
R_scl = np.array([[2, 0], [0, 0.5]])
R_shr = np.array([[1, 0.5], [0, 1]])

transforms = [
    (R_rot, 'Rotation 45°'),
    (R_scl, 'Scaling (2, 0.5)'),
    (R_shr, 'Shear'),
]

fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, (M, title) in zip(axes, transforms):
    transformed = M @ square
    ax.fill(square[0], square[1], alpha=0.3, color='steelblue', label='Original')
    ax.plot(square[0], square[1], 'steelblue', linewidth=2)
    ax.fill(transformed[0], transformed[1], alpha=0.3, color='crimson', label=
↪'Transformed')
    ax.plot(transformed[0], transformed[1], 'crimson', linewidth=2)
    ax.set_xlim(-1.5, 2.5)
    ax.set_ylim(-1.5, 2.0)
    ax.set_aspect('equal')
    ax.set_title(title)
    ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)

plt.suptitle('Linear transformations of the unit square', y=1.02)
```

(continues on next page)

(continued from previous page)

```
plt.tight_layout()
plt.show()
```

11.4.1 Interpretation

- **Rotation:** preserves areas and angles (orthogonal transformation, $\det = 1$).
- **Scaling:** stretches one axis and compresses another ($\det = 2 \times 0.5 = 1$).
- **Shear:** shifts one axis proportionally to the other ($\det = 1$, preserves area).

```
for M, name in transforms:
    print(f'{name}:  det = {np.linalg.det(M):.2f}')
```

11.4.2 Conclusion

The matrix determinant tells us whether the transformation preserves area ($|\det| = 1$), expands it ($|\det| > 1$), or contracts it ($|\det| < 1$). Visualizing the effect on the unit square is the most intuitive way to understand any linear transformation.

CHAPTER

TWELVE

DEGREE IN STATISTICS

Examples oriented towards data analysis and probability. Highlights:

- DataFrame handling (Pandas)
- Statistical visualization (Seaborn)
- Probabilistic models

12.1 Statistics Example: Data Generation

We generate a normal distribution and visualize its histogram.

```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.normal(0, 1, 1000)

plt.hist(data, bins=30, alpha=0.7, color='green', density=True)
plt.title('Normal Distribution Histogram')
plt.xlabel('Values')
plt.ylabel('Density')
plt.show()
```

12.2 Linear Regression with Synthetic Data

We generate bivariate data with an underlying linear relationship and Gaussian noise. We fit a linear regression model using least squares and evaluate the fit quality with R^2 and residual analysis.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(12)
n = 50
x = np.linspace(2, 12, n)
y_true = 3.5 * x + 10
y = y_true + np.random.normal(0, 5, n)

coef = np.polyfit(x, y, 1)
y_fit = np.polyval(coef, x)
residuals = y - y_fit
```

(continues on next page)

(continued from previous page)

```
SS_res = np.sum(residuals**2)
SS_tot = np.sum((y - np.mean(y))**2)
R2 = 1 - SS_res / SS_tot

print(f'Fitted equation: y = {coef[0]:.3f}x + {coef[1]:.3f}')
print(f'R² = {R2:.4f}')
```

12.2.1 Scatter plot with regression line

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

axes[0].scatter(x, y, alpha=0.7, color='steelblue', edgecolors='navy', label='Data')
axes[0].plot(x, y_fit, 'r-', linewidth=2, label=f'y = {coef[0]:.2f}x + {coef[1]:.2f}')
axes[0].set_xlabel('x')
axes[0].set_ylabel('y')
axes[0].set_title(f'Linear Regression (R² = {R2:.3f})')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].scatter(y_fit, residuals, alpha=0.7, color='darkorange', edgecolors='brown')
axes[1].axhline(0, color='gray', linewidth=0.8, linestyle='--')
axes[1].set_xlabel('Fitted values')
axes[1].set_ylabel('Residuals')
axes[1].set_title('Residual plot')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

12.2.2 Conclusion

The residual plot allows us to verify regression assumptions: if residuals are randomly scattered around zero, the linear model is adequate. A pattern in the residuals would indicate a missing nonlinear term.

12.3 Estimating π with Monte Carlo

The Monte Carlo method estimates π by throwing random points in a unit square $[0, 1] \times [0, 1]$. The fraction that falls inside the quarter circle (radius 1) approximates $\pi/4$:

$$\pi \approx 4 \cdot \frac{N_{\text{inside}}}{N_{\text{total}}}$$

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
N = 5000
x = np.random.uniform(0, 1, N)
y = np.random.uniform(0, 1, N)
```

(continues on next page)

(continued from previous page)

```
inside = x**2 + y**2 <= 1
pi_est = 4 * np.sum(inside) / N

print(f'Total points:     {N}')
print(f'Inside circle:    {np.sum(inside)}')
print(f'π estimate:       {pi_est:.6f}')
print(f'True π value:     {np.pi:.6f}')
print(f'Absolute error:   {abs(pi_est - np.pi):.6f}')
```

12.3.1 Point Visualization

Blue points fall inside the quarter circle, red points outside. The black curve is the boundary $x^2 + y^2 = 1$.

```
plt.figure(figsize=(6, 6))
plt.scatter(x[inside], y[inside], s=1, color='steelblue', alpha=0.5, label='Inside')
plt.scatter(x[~inside], y[~inside], s=1, color='crimson', alpha=0.3, label='Outside')
theta = np.linspace(0, np.pi/2, 200)
plt.plot(np.cos(theta), np.sin(theta), 'k-', linewidth=2)
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.gca().set_aspect('equal')
plt.title(f'Monte Carlo — π ≈ {pi_est:.4f}  ({N} points)')
plt.legend(markerscale=10, fontsize=9)
plt.tight_layout()
plt.show()
```

12.3.2 Convergence as N Increases

The estimate improves as we increase the number of points. The error decreases as $O(1/\sqrt{N})$, characteristic of Monte Carlo methods.

12.4 Practice Questions: Statistics

Test your knowledge of fundamental statistics concepts. Expand each question to reveal the answer.

Question 1 — p-value

What exactly does a p-value of 0.03 mean in a hypothesis test?

A p-value of 0.03 means that, **if the null hypothesis H_0 were true**, the probability of observing a result as extreme or more extreme than the one obtained would be 3%.

Important: It is NOT the probability that H_0 is true. It is the probability of the data (or more extreme) given H_0.

Since 0.03 < 0.05 (the typical significance level), we would reject H_0.

Question 2 — Confidence interval

Interpret a 95% confidence interval for a mean: $[12.3, 15.7]$**.**

It does NOT mean the population mean has a 95% probability of being in that interval. The correct frequentist interpretation is:

> If we repeat the experiment many times, 95% of the intervals constructed with the same method will contain the true parameter value.

For this particular interval, the population mean is either inside or outside — there is no intermediate probability. What has 95% confidence is the **construction method**.

Question 3 — Normal distribution

What proportion of data falls within ±2 standard deviations of the mean in a normal distribution?

Using the empirical rule (68-95-99.7 rule):

The following table summarizes the main elements of this section.

Table. Practice Questions: Statistics.

Range	Proportion
$\mu \pm 1\sigma$	$\approx 68.27\%$
$\mu \pm 2\sigma$	$\approx 95.45\%$
$\mu \pm 3\sigma$	$\approx 99.73\%$

Therefore, approximately **95.45%** of data falls within $\mu \pm 2\sigma$.

Question 4 — Correlation vs. Causation

If two variables have a correlation of $r = 0.85$**, can you conclude that one causes the other?**

No. Correlation does not imply causation. A strong correlation only indicates linear association. Possible explanations include:

1. X **causes** Y — possible but not proven by r alone.
2. Y **causes** X — the direction could be reversed.
3. **A confounding variable** Z causes both.
4. **Coincidence** — especially with small datasets.

Establishing causation requires experimental studies (randomized controlled trials), not just observational data.

Question 5 — Type I and Type II Errors

Define Type I and Type II errors. Which one is directly controlled by the α level?

The following table summarizes the main elements of this section.

Table. Practice Questions: Statistics.

Type	Occurs when...	Called
Type I Error	We reject H_0 when it is true	False positive
Type II Error	We fail to reject H_0 when it is false	False negative

The **significance level** α directly controls the probability of committing a Type I error: $P(\text{Type I}) = \alpha$.

The probability of a Type II error is denoted β, and the power of a test is $1 - \beta$. Decreasing α generally increases β.

Question 6 — Central Limit Theorem

State the Central Limit Theorem and explain its practical importance.

The **Central Limit Theorem (CLT)** states that, for sufficiently large samples ($n \geq 30$, approximately), the distribution of the sample mean $\bar{X}$ approaches a normal distribution:

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

regardless of the original population distribution.

Practical importance: It justifies using the normal distribution to construct confidence intervals and perform hypothesis tests about the mean, even when the population is not normal. It is the foundation of much of inferential statistics.

CHAPTER

THIRTEEN

DEGREE IN COMPUTER SCIENCE

In this section, you will find examples adapted to Computer Science Degree courses. The goal is to show how to integrate:

- Database diagrams (E-R model)
- UML class diagrams
- Algorithmic complexity analysis with executable code

Applicable subjects

This block can be adapted to programming, databases, and software engineering courses.

13.1 Entity-Relationship Diagram with Kroki (Mermaid)

E-R diagrams are fundamental in database design. With Kroki and `:type: mermaid`, you can include them directly in your book and make them work in both HTML and PDF.

13.1.1 Example: University database

Code:

```
```{kroki}
:type: mermaid
:align: center

erDiagram
    STUDENT {
        int id PK
        string name
        string email
    }
    COURSE {
        int id PK
        string name
        int credits
    }
    PROFESSOR {
        int id PK
        string name
        string department
```
```

(continues on next page)

(continued from previous page)

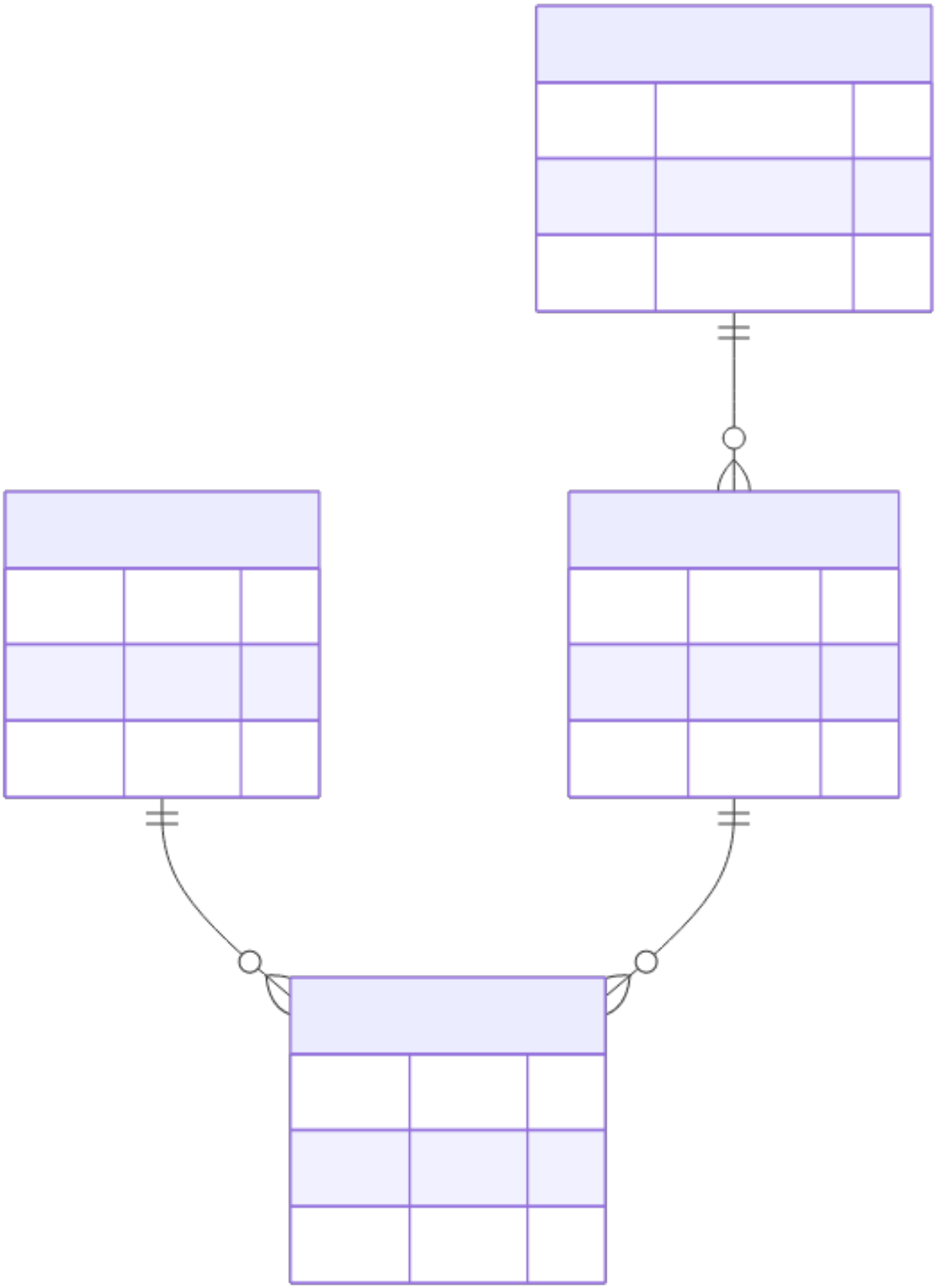
```
    }
    ENROLLMENT {
        int id PK
        float grade
        string term
    }

    STUDENT ||--o{ ENROLLMENT : "enrolls in"
    COURSE ||--o{ ENROLLMENT : "has"
    PROFESSOR ||--o{ COURSE : "teaches"
```

Result:

The following diagram visually summarizes this part of the explanation.

Diagram: Example: University database.



13.1.2 How to modify it

- **Add an entity**: Copy a block like `ENTITY { ... }` and define its attributes.
- **Relationship types**: Use `||--o{` (one-to-many), `||--||` (one-to-one), `}o--o{` (many-to-many).
- **Primary keys**: Mark attributes with `PK`.
- **Foreign keys**: Mark attributes with `FK`.

13.1.3 Cardinality reference

The following table summarizes the main elements of this section.

Table. Cardinality reference.

Notation	Meaning
`	
`	
`}o--o{`	Many to many

13.2 UML Class Diagram with Kroki (Mermaid)

Class diagrams are essential for modeling object-oriented systems. With Kroki and `:type: mermaid`, you can represent classes, attributes, methods, and inheritance in both HTML and PDF.

13.2.1 Example: Geometric Shape Hierarchy

Code:

```
```{kroki}
:type: mermaid
:align: center

classDiagram
    class Shape {
        <<abstract>>
        +String color
        +area() float
        +perimeter() float
    }

    class Circle {
        +float radius
        +area() float
        +perimeter() float
    }

    class Rectangle {
        +float width
```
```

(continues on next page)

(continued from previous page)

```
        +float height
        +area() float
        +perimeter() float
    }

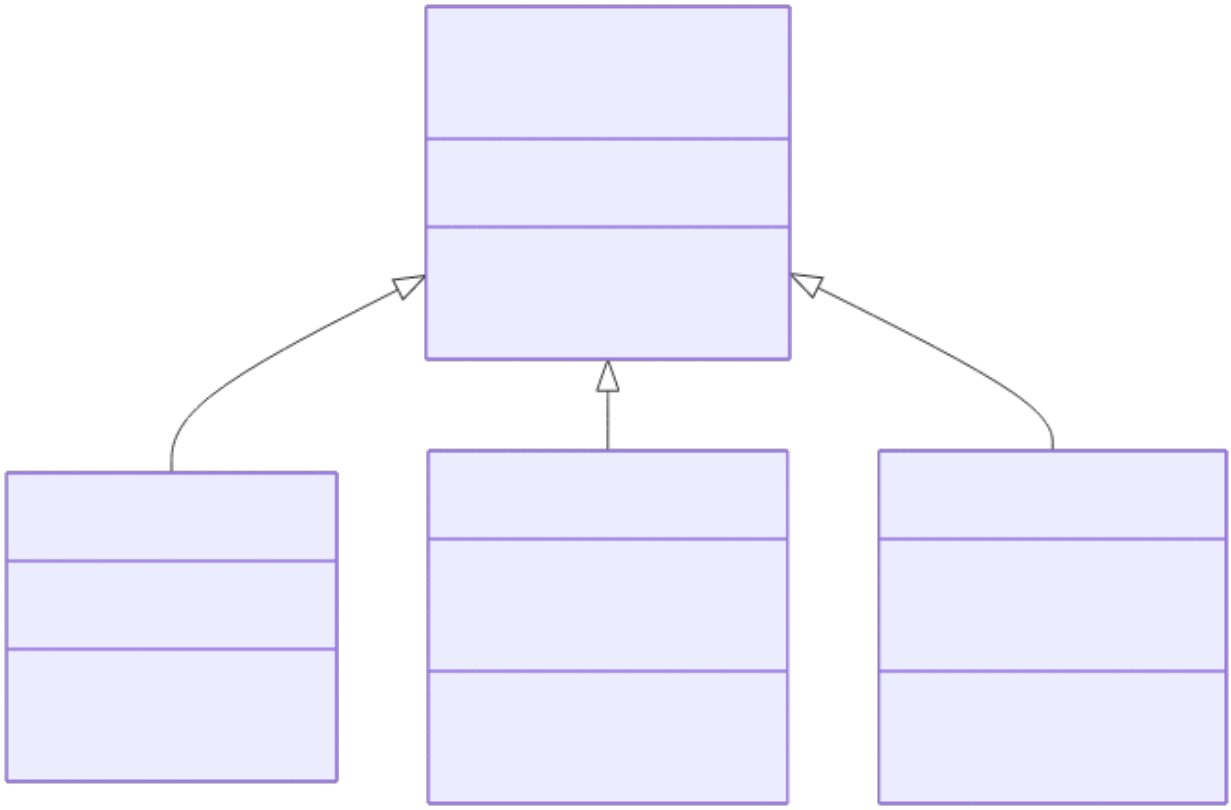
    class Triangle {
        +float base
        +float height
        +area() float
        +perimeter() float
    }

    Shape <|-- Circle
    Shape <|-- Rectangle
    Shape <|-- Triangle
```

Result:

The following diagram visually summarizes this part of the explanation.

Diagram: Example: Geometric Shape Hierarchy.



13.2.2 Class notation

- **Attributes**: Listed in the first section. Prefix with + (public), − (private), # (protected).
- **Methods**: Listed in the second section, with return type.
- **Abstract class**: Indicated with `<<abstract>>`.

13.2.3 Relationship types

The following table summarizes the main elements of this section.

Table. Relationship types.

Notation	Meaning
`<|--`	Inheritance (is a)
`*--`	Composition
`o--`	Aggregation
`-->`	Association
`..>`	Dependency

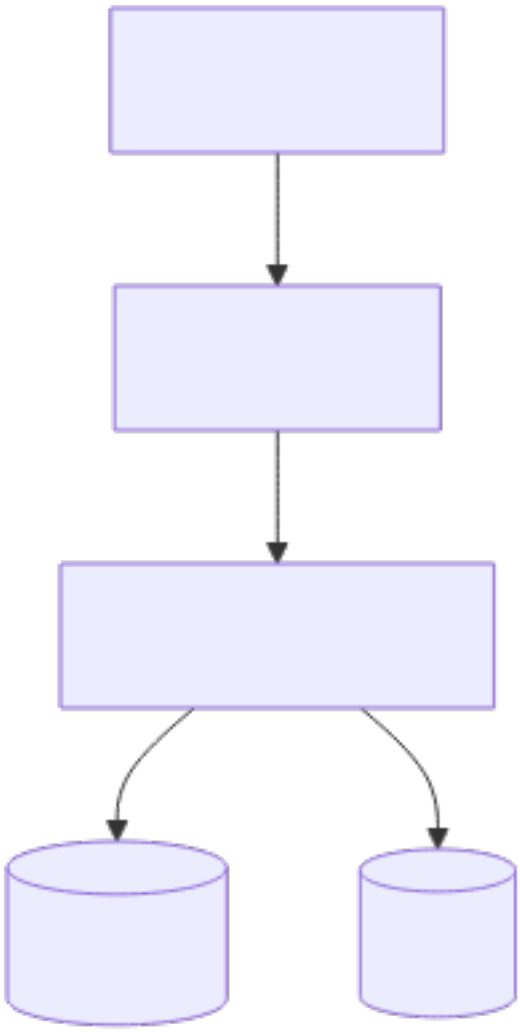
13.3 Diagrams with Kroki for Computer Science

Kroki lets you generate many useful diagram types in Computer Science without depending on browser-side JavaScript. All the following examples work in **HTML and PDF**.

13.3.1 1. Mermaid: layered architecture

The following diagram visually summarizes this part of the explanation.

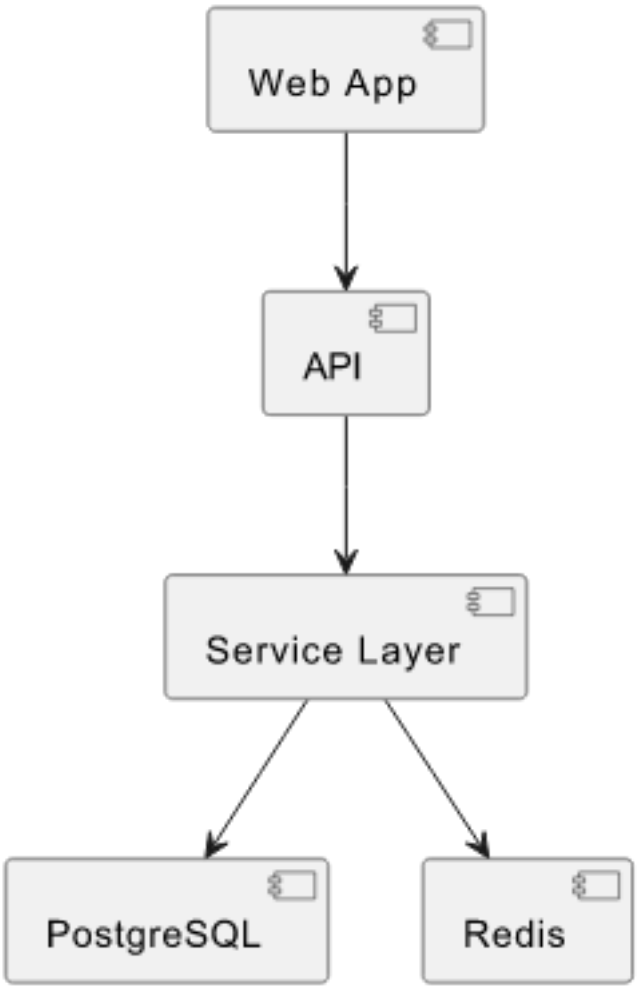
Diagram: Mermaid: layered architecture.



13.3.2 2. PlantUML: system components

The following diagram visually summarizes this part of the explanation.

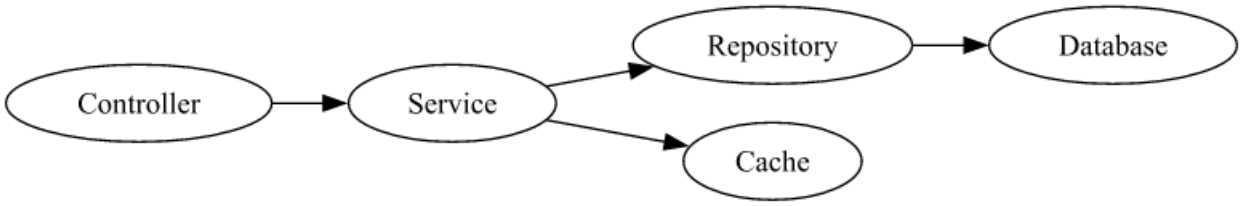
Diagram: PlantUML: system components.



13.3.3 3. GraphViz: dependency graph

The following diagram visually summarizes this part of the explanation.

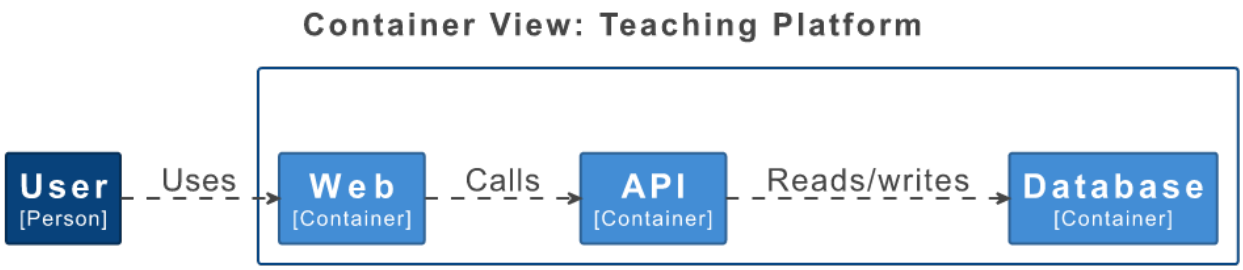
Diagram: GraphViz: dependency graph.



13.3.4 4. Structurizr: simplified C4 view

The following diagram visually summarizes this part of the explanation.

Diagram: Structurizr: simplified C4 view.



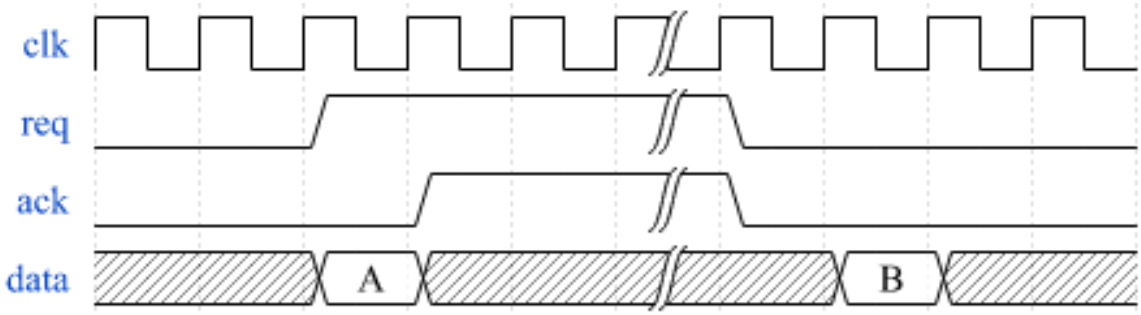
PDF note

For C4 diagrams, define colors explicitly. Some default themes may produce white text on a white background when SVG is converted to PDF.

13.3.5 5. Wavedrom: digital protocol timing

The following diagram visually summarizes this part of the explanation.

Diagram: Wavedrom: digital protocol timing.



13.3.6 Key idea

In Computer Science, Kroki is ideal for:

- architecture
- UML
- dependency graphs
- protocols
- C4 diagrams

If the user asks for a fast and portable diagram, Kroki is a great option.

13.4 Algorithmic Complexity: Visual Comparison

How does execution time grow as we increase data size? Let's compare three classic complexities:

- $O(n)$ — linear
- $O(n \log n)$ — linearithmic
- $O(n^2)$ — quadratic

```
import numpy as np
import matplotlib.pyplot as plt
import time

def measure_time(func, data):
    start = time.perf_counter()
    func(data)
    return time.perf_counter() - start

def linear_search(lst):
    target = lst[-1]
    for x in lst:
        if x == target:
            return x

def sort_data(lst):
    return sorted(lst)

def bubble_sort(lst):
    n = len(lst)
    for i in range(n):
        for j in range(0, n - i - 1):
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]

sizes = [100, 500, 1000, 2000, 3000, 5000]
t_on, t_onlogn, t_on2 = [], [], []

for n in sizes:
    data = list(range(n, 0, -1))
    t_on.append(measure_time(linear_search, data[:]))
    t_onlogn.append(measure_time(sort_data, data[:]))
    t_on2.append(measure_time(bubble_sort, data[:]))

plt.figure(figsize=(8, 5))
plt.plot(sizes, t_on, 'o-', label='O(n) — Linear search')
plt.plot(sizes, t_onlogn, 's-', label='O(n log n) — Sorting')
plt.plot(sizes, t_on2, '^-', label='O(n²) — Bubble sort')
plt.xlabel('Input size (n)')
plt.ylabel('Time (s)')
plt.title('Algorithmic Complexity Comparison')
plt.legend()
plt.grid(True)
plt.show()
```

13.4.1 Expected result

The plot clearly shows how $O(n^2)$ grows much faster than $O(n)$ and $O(n \log n)$. This explains why choosing the right algorithm is crucial when working with large datasets.

13.4.2 Conclusion

- $O(n)$: Time grows proportionally to data size.
- $O(n \log n)$: Moderate growth, typical of efficient sorting algorithms.
- $O(n^2)$: Explosive growth, unfeasible for large data.

Choosing a good algorithm can mean the difference between seconds and hours of computation.

CHAPTER

FOURTEEN

DEGREE IN GEOLOGY

In this section, you will find examples adapted to Geology Degree courses. The goal is to show how to integrate:

- Topographic profiles from elevation data
- Stratigraphic columns with executable code
- Rock cycle diagrams

Applicable subjects

This block can be adapted to cartography, stratigraphy, general geology, geomorphology, and petrology courses.

14.1 Topographic Profile

A topographic profile represents elevation variation along a section line. We will generate a synthetic profile and plot it with matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

distance = np.linspace(0, 10, 500)

elevation = (
    200
    + 80 * np.sin(0.8 * distance)
    + 40 * np.sin(2.5 * distance)
    + 20 * np.random.normal(0, 1, len(distance))
)

plt.figure(figsize=(10, 4))
plt.fill_between(distance, 0, elevation, color='#8B7355', alpha=0.4)
plt.plot(distance, elevation, color='#5C4033', linewidth=1.5)
plt.xlabel('Distance (km)')
plt.ylabel('Elevation (m)')
plt.title('Synthetic topographic profile')
plt.axhline(y=0, color='skyblue', linewidth=0.5)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

14.1.1 Expected result

A profile simulating terrain with elevation variations is produced. The brown fill represents the ground mass and the blue line marks sea level.

14.1.2 Conclusion

- Topographic profiles can be created from real data (CSV, GeoTIFF).
- This example uses synthetic data to avoid external file dependencies.
- For real data, simply replace the `distance` and `elevation` arrays.

14.2 Stratigraphic Column

A stratigraphic column represents the vertical succession of geological layers. Each layer is drawn as a colored rectangle according to its lithology.

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

layers = [
    {"name": "Limestone",     "thickness": 15, "color": "#4A90D9", "age": "Jurassic"},
    {"name": "Sandstone",     "thickness": 20, "color": "#E8C84A", "age": "Triassic"},
    {"name": "Clay",          "thickness": 10, "color": "#C97B3D", "age": "Triassic"},
    {"name": "Conglomerate",  "thickness": 12, "color": "#9B7653", "age": "Permian"},
    {"name": "Shale",         "thickness": 18, "color": "#6B6B6B", "age":
 ↪"Carboniferous"},
    {"name": "Granite",       "thickness": 25, "color": "#D44A4A", "age": "Precambrian
 ↪"},
]

fig, ax = plt.subplots(figsize=(5, 8))

y_base = 0
for layer in layers:
    ax.add_patch(plt.Rectangle((0, y_base), 1, layer["thickness"],
                               facecolor=layer["color"], edgecolor="black",␣
 ↪linewidth=0.8))
    ax.text(0.5, y_base + layer["thickness"] / 2,
            f"{layer['name']}\n({layer['age']}, {layer['thickness']} m)",
            ha='center', va='center', fontsize=8, fontweight='bold')
    y_base += layer["thickness"]

ax.set_xlim(-0.2, 1.4)
ax.set_ylim(-2, y_base + 2)
ax.set_ylabel('Cumulative thickness (m)')
ax.set_title('Synthetic stratigraphic column')
ax.set_xticks([])
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```

14.2.1 Expected result

The plot shows stacked layers from bottom to top (oldest to youngest). Each color represents a different lithology: blue for limestone, yellow for sandstone, etc.

14.2.2 Conclusion

- You can add more layers by appending entries to the `layers` list.
- Colors are customizable by changing the hex values.
- For real data, replace the list with data read from a CSV file.

14.3 The Rock Cycle with Kroki (Mermaid)

The rock cycle describes the transformations between the three main rock types. With Kroki and `:type: mermaid`, we can represent this cycle as a flowchart that works in both HTML and PDF.

14.3.1 The cycle

Code:

```
```{kroki}
:type: mermaid
:align: center

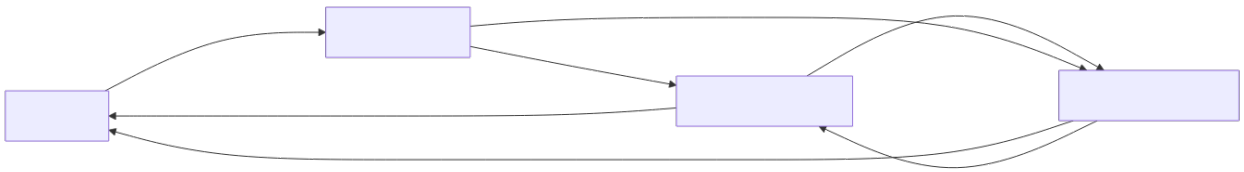
flowchart LR
    Magma["Magma"]
    Igneous["Igneous Rock"]
    Sedimentary["Sedimentary Rock"]
    Metamorphic["Metamorphic Rock"]

    Magma -->|"Cooling and solidification"| Igneous
    Igneous -->|"Weathering and erosion"| Sedimentary
    Sedimentary -->|"Heat and pressure"| Metamorphic
    Metamorphic -->|"Melting"| Magma
    Igneous -->|"Heat and pressure"| Metamorphic
    Metamorphic -->|"Weathering and erosion"| Sedimentary
    Sedimentary -->|"Melting"| Magma
```
```

Result:

The following diagram visually summarizes this part of the explanation.

Diagram: The cycle.



14.3.2 Transition explanation

The following table summarizes the main elements of this section.

Table. Transition explanation.

Transition	Geological process
Magma → Igneous	Cooling and solidification at the surface or underground
Igneous → Sedimentary	Weathering, erosion, transport, and sedimentation
Sedimentary → Metamorphic	Heat and pressure (metamorphism) without melting
Metamorphic → Magma	Complete melting at extreme temperatures
Igneous → Metamorphic	Direct metamorphism through heat and pressure
Metamorphic → Sedimentary	Uplift, weathering, and erosion

14.3.3 Key point

Any rock type can transform into any other. It is not a linear cycle, but a **transformation network** operating on geological timescales (millions of years).

Part III

Cross-disciplinary Examples

CHAPTER

FIFTEEN

CROSS-DISCIPLINARY EXAMPLES

What are cross-disciplinary examples?

They are reusable teaching patterns for many degrees, not examples locked to a single discipline. The idea is to learn a technique —modeling a process, analyzing a survey, building a rubric, summarizing texts, simulating data, or solving a balance— and then adapt it to sciences, humanities, social sciences, health, engineering, or education.

15.1 Example families

The following table summarizes the main elements of this section.

Table. Example families.

Example	Cross-disciplinary pattern	Can be adapted to...
Growth and saturation models	Systems that grow toward a limit	Populations, cultural diffusion, technology adoption, learning, epidemics
Questionnaire analysis	Surveys and opinion scales	Psychology, education, sociology, teaching satisfaction, market studies
Text corpus frequencies	Counting, classifying, and summarizing terms	Languages, history, law, communication, document analysis, NLP
Rubrics and learning sequences	Transparent assessment and planning	Any course with essays, labs, projects, or presentations
Simulated data and variable relationships	Data exploration and correlations	Health, economics, education, environment, social sciences, laboratory work
Balances and conservation	Inputs, outputs, accumulation, and transformation	Chemistry, ecology, economics, logistics, energy, budgets, demography

How to use this section

Do not copy the discipline: copy the **pattern**. Change the vocabulary, the data, and the teaching question so it fits your course. That is where the cross-disciplinary value lives.

15.2 The common idea

All these examples follow the same philosophy:

1. describe a problem with a clear structure;
2. represent data, rules, or criteria as code or tables;
3. generate a reproducible teaching artifact;
4. allow AI to help adapt the example to another context.

This turns each page into a template: it is not "a Biology example" or "a Psychology example", but a recipe that can travel across fields.

15.3 Growth and Saturation Models

The logistic model describes any process that grows quickly at first and slows down as it approaches a limit: a population, technology adoption, diffusion of an idea, accumulated learning, or the early spread of a disease.

$$\frac{dN}{dt} = rN\left(1 - \frac{N}{K}\right)$$

- N = accumulated quantity of the phenomenon
- r = intrinsic growth rate
- K = limit or maximum capacity of the system

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

def logistic(N, t, r, K):
    return r * N * (1 - N / K)

t = np.linspace(0, 50, 500)
N0 = 10

params = [(0.1, 500), (0.3, 500), (0.3, 1000)]
labels = ['r=0.1, K=500', 'r=0.3, K=500', 'r=0.3, K=1000']

fig, ax = plt.subplots(figsize=(8, 5))
for (r, K), label in zip(params, labels):
    sol = odeint(logistic, N0, t, args=(r, K))
    ax.plot(t, sol, label=label)

ax.set_xlabel('Time')
ax.set_ylabel('Population (N)')
ax.set_title('Logistic growth curves')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

15.3.1 Applications

- **Biology**: population dynamics, bacterial growth
- **Medicine**: epidemic spread (simplified SIR model)
- **Economics**: adoption of new technologies

Try it yourself

Change the values of `r` and `K` in the `params` list and observe how the curve changes. What happens when `r` is very high?

15.4 Questionnaire Analysis

We generate synthetic data from a Likert-scale survey (1-5) and analyze the results. The same pattern works for teaching satisfaction, classroom climate, public opinion, market studies, or self-assessment.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

n_participants = 120
questions = [
    'Clarity of explanations',
    'Teaching materials',
    'Practical usefulness',
    'Pace of the class',
    'Teacher availability',
    'Fair assessment',
    'Class atmosphere',
    'Workload',
    'Online resources',
    'Would recommend the course'
]

data = {}
for q in questions:
    center = np.random.choice([2.5, 3.0, 3.5, 4.0])
    data[q] = np.clip(np.round(np.random.normal(center, 0.8, n_participants)), 1, 5).
 ↪astype(int)

df = pd.DataFrame(data)
df.head(10)
```

```
summary = df.agg(['mean', 'std']).T.round(2)
summary.columns = ['Mean', 'Std Deviation']
summary
```

```
fig, ax = plt.subplots(figsize=(10, 6))

frequencies = df.apply(lambda col: col.value_counts().reindex(range(1, 6), fill_
```

(continues on next page)

(continued from previous page)

```
↪value=0))
colors = ['#d73027', '#fc8d59', '#fee08b', '#91bfdb', '#4575b4']

x = np.arange(len(questions))
width = 0.15

for i, (val, color) in enumerate(zip(range(1, 6), colors)):
    ax.bar(x + i * width, frequencies.loc[val], width, label=str(val), color=color)

ax.set_xticks(x + 2 * width)
ax.set_xticklabels([q[:25] + '...' if len(q) > 25 else q for q in questions],␣
↪rotation=45, ha='right')
ax.set_ylabel('Number of responses')
ax.set_title('Response distribution by question')
ax.legend(title='Value', bbox_to_anchor=(1.05, 1))
plt.tight_layout()
plt.show()
```

15.4.1 Applications

- **Psychology**: attitude and perception analysis
- **Education**: student satisfaction evaluation
- **Medicine**: quality of life surveys

Adapt this example

Replace the `questions` list with your own survey items and adjust `n_participants`. The code generates data and plots automatically.

15.5 Text Corpus Frequencies

In any discipline that works with documents, **term frequency** indicates how many times each word or concept appears in a corpus. It helps detect dominant topics, key vocabulary, and shifts in focus across texts.

Why does frequency matter?

The most frequent words or concepts often reveal the structure of a discourse. The same technique can be applied to novels, legal decisions, interviews, scientific articles, historical newspapers, or open-ended survey responses.

15.5.1 Frequency table: sample corpus

The following table summarizes the main elements of this section.

Table. Frequency table: sample corpus.

Term	Grammatical category	Frequency	Cumulative percentage
the	Definite article	6 892	12.3 %
of	Preposition	5 417	22.0 %
and	Conjunction	4 756	30.5 %
to	Preposition	4 102	37.8 %
a	Indefinite article	3 945	44.8 %
in	Preposition	3 401	50.9 %
is	Verb	2 980	56.2 %
it	Pronoun	2 654	60.9 %
for	Preposition	2 432	65.3 %
that	Conjunction / Pronoun	2 180	69.2 %

15.5.2 Teaching applications

- **Humanities**: topic analysis in literary, historical, or philosophical texts
- **Social sciences**: study of interviews, open-ended surveys, or public discourse
- **Health sciences**: review of frequent terms in reports or simulated anamnesis
- **Engineering and computer science**: foundations of text mining and NLP
- **Education**: design of materials graded by lexical or conceptual difficulty

How to adapt this table

Replace the frequencies with those from your own corpus. Tools such as AntConc, Voyant Tools, or Python compute frequencies automatically from almost any collection of texts.

15.6 Rubrics and Learning Sequences

15.6.1 Rubric: Research project

The following table summarizes the main elements of this section.

Table. Rubric: Research project.

Criterion	Excellent (4)	Good (3)	Sufficient (2)	Insufficient (1)
Problem statement	Defines an original, relevant problem with clear context	Defines an adequate problem with some context	Defines a vague or poorly contextualized problem	No identifiable problem defined
Methodology	Rigorous, well-justified, replicable method	Adequate method with partial justification	Simple method without clear justification	No defined method
Data analysis	In-depth analysis with correct interpretation	Correct analysis with basic interpretation	Superficial analysis with minor errors	No analysis or major errors
Conclusions	Well-founded conclusions answering the problem	Coherent conclusions aligned with data	Partial or poorly connected conclusions	No conclusions or unrelated to the work
Presentation	Impeccable structure, clear writing, no errors	Good structure, minimal errors	Acceptable structure, some errors	Disorganized, frequent errors

Scoring

Minimum passing score: **10 / 20** (50 %). The total score is obtained by adding the values of each criterion.

15.6.2 Learning objective sequence

The following table summarizes the main elements of this section.

Table. Learning objective sequence.

Phase	Objective	Expected outcome
1. Exploration	Identify a topic of interest	List of 3 candidate topics
2. Delimitation	Formulate a research question	Concrete, viable question
3. Design	Select an appropriate methodology	Research protocol
4. Collection	Gather relevant data	Organized dataset
5. Analysis	Interpret the results	Findings report
6. Communication	Present the final work	Written document + oral defense

15.6.3 Cross-disciplinary applications

- **Humanities**: assessment of essays, text commentaries, and presentations
- **Social sciences**: assessment of case studies, reports, and debates
- **Experimental sciences**: assessment of lab work and field notebooks
- **Engineering**: assessment of projects, prototypes, and technical documentation
- **Health**: assessment of simulated clinical cases and diagnostic reasoning

Adapt the rubric

Replace the criteria in the first column with those from your course and adjust the descriptors. The structure works for almost any assessable product: essay, lab activity, report, project, poster, presentation, or portfolio.

15.7 Simulated Data and Variable Relationships

We generate a simulated dataset and explore correlations between variables. Although the example uses clinical variables, the pattern works for economics, education, environment, laboratory work, or social sciences.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)
n = 200

age = np.random.randint(30, 80, n)
cholesterol = 150 + 1.5 * age + np.random.normal(0, 25, n)
systolic_bp = 110 + 0.5 * age + np.random.normal(0, 10, n)
risk = (0.3 * (age - 30) + 0.1 * (cholesterol - 150) + np.random.normal(0, 5, n)).
↪clip(0, 100)

df = pd.DataFrame({
    'Age': age,
    'Cholesterol (mg/dL)': cholesterol.round(0),
    'Systolic BP (mmHg)': systolic_bp.round(0),
    'Cardiovascular risk (%)': risk.round(1)
})

df.describe().round(1)
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].scatter(df['Age'], df['Cholesterol (mg/dL)'], alpha=0.5, c='steelblue')
axes[0].set_xlabel('Age (years)')
axes[0].set_ylabel('Cholesterol (mg/dL)')
axes[0].set_title('Age vs Cholesterol')

scatter = axes[1].scatter(
    df['Cholesterol (mg/dL)'], df['Cardiovascular risk (%)'],
    alpha=0.5, c=df['Age'], cmap='RdYlBu_r'
)
axes[1].set_xlabel('Cholesterol (mg/dL)')
axes[1].set_ylabel('Cardiovascular risk (%)')
axes[1].set_title('Cholesterol vs Risk (color = Age)')
plt.colorbar(scatter, ax=axes[1], label='Age')

plt.tight_layout()
plt.show()
```

```
correlations = df.corr().round(2)
correlations
```

15.7.1 Applications

- **Medicine**: exploratory analysis of clinical data
- **Psychology**: correlation between well-being variables
- **Biology**: relationship between physiological variables

Important note

These data are **simulated** and do not represent real patients. In actual clinical research, always verify the origin and quality of the data.

15.8 Balances and Conservation

15.8.1 Problem statement

A solution of reactant A with concentration $C_{A0} = 2.0$ mol/L is fed into a continuous stirred-tank reactor (CSTR) at a volumetric flow rate $Q = 10$ L/min. The reaction is:

$$A \longrightarrow B \quad \text{with} \quad r_A = -k\,C_A$$

where $k = 0.5$ min^{-1}. The reactor volume is $V = 100$ L.

Determine the outlet concentration C_A.

15.8.2 Mass balance equation

The steady-state mass balance for component A is:

$$0 = Q\,C_{A0} - Q\,C_A + r_A\,V$$

Substituting the reaction rate expression:

$$0 = Q\,C_{A0} - Q\,C_A - k\,C_A\,V$$

15.8.3 Step-by-step solution

Step 1: Group terms with C_A.

$$Q\,C_{A0} = C_A\,(Q + kV)$$

Step 2: Solve for C_A.

$$C_A = \frac{Q\,C_{A0}}{Q + kV}$$

Step 3: Substitute numerical values.

$$C_A = \frac{10 \times 2.0}{10 + 0.5 \times 100} = \frac{20}{60} = 0.333 \text{ mol/L}$$

15.8.4 Conversion

The reactor conversion is defined as:

$$X = 1 - \frac{C_A}{C_{A0}} = 1 - \frac{0.333}{2.0} = 0.833 \ (83.3\%)$$

Quick verification

Residence time: $\tau = V/Q = 100/10 = 10$ min. For a first-order CSTR: $X = k\tau/(1 + k\tau) = 5/6 = 0.833$. The results match.

15.8.5 Cross-disciplinary applications

- **Chemical Engineering**: reactor design, stoichiometric calculations
- **Biology and environment**: bioreactor, ecosystem, and wastewater treatment models
- **Medicine**: pharmacokinetics and compartment models
- **Economics**: income, expenses, debt, and accumulation flows
- **Logistics**: inputs, outputs, and material storage
- **Energy**: power balances, consumption, and losses

Part IV

Basic Content

CHAPTER

SIXTEEN

BASIC CONTENT

This section is your **quick reference** for the most common TeachBook elements. Each page shows the MyST code you need to write and explains what it produces when compiled.

> **Note**
>
> All examples in this section work in both **HTML** and **PDF**, unless explicitly stated otherwise.

16.1 What you will find here

The following table summarizes the main elements of this section.

Table. What you will find here.

Page	What you will learn
Figures and images	Inserting and referencing images
Equations	Inline and display math
Citations and bibliography	BibTeX references
Cross-references	Linking figures, tables, sections…
Tables	Simple and formatted tables
Admonitions and dropdowns	Notes, warnings, hints, collapsible blocks
Tabs and cards	Tabbed content (sphinx-design)
Source code	Code blocks with syntax highlighting
Videos	YouTube and local video (HTML + PDF)
Mermaid diagrams	Flowcharts, sequences, class diagrams…
HTML/PDF compatibility	What works where and how to ensure compatibility

16.2 How to use this section

1. Find the page you need.
2. Copy the example that closest matches what you want.
3. Adapt the content to your case.
4. Build and verify the result.

Tip

If you use an AI assistant, you can ask it to "generate a figure like the one in the *Figures* section but with my image" and save time.

CHAPTER

SEVENTEEN

FIGURES AND IMAGES

Figures are essential in any teaching material. MyST offers two main directives: `{image}` (simple image) and `{figure}` (image with number and caption).

17.1 Centered image with alt text

The `{image}` directive inserts an image without numbering:

```
```{image} _static/logo.png
:alt: TeachBook project logo
:width: 60%
:align: center
```
```

```
Result:

```{image} _static/logo.png
:alt: TeachBook project logo
:width: 60%
:align: center
```
```

17.2 Numbered figure with caption

The `{figure}` directive adds automatic numbering (`Figure 1`, `Figure 2`...):

```
```{figure} _static/logo.png
:width: 50%
:align: center
:name: fig-figures-1

Official TeachBook project logo.
```
```

Result:

The Fig. 17.1 visually summarizes this part of the explanation.

Fig. 17.1: Official TeachBook project logo.

17.3 Cross-reference to a figure

Use `{numref}` to reference figures by their number:

```
The {numref}`fig-logo-en` shows the project logo.
```

Result: The Fig. 17.1 shows the project logo.

17.4 Margin figure

Add `:figclass: margin` to place an image in the margin (HTML only):

```
```{figure} _static/logo.png
:figclass: margin
:width: 100%
:name: fig-figures-3

Logo in the margin.
```
```

Result:

The Fig. 17.2 visually summarizes this part of the explanation.

Fig. 17.2: Logo in the margin.

17.5 Useful parameters

The following table summarizes the main elements of this section.

Table. Useful parameters.

Parameter	What it does	Example
`:width:`	Width (`50%`, `200px`)	`:width: 80%`
`:align:`	Alignment (`left`, `center`, `right`)	`:align: center`
`:alt:`	Alt text (accessibility)	`:alt: Diagram`
`:name:`	Label for `{numref}`	`:name: fig-map`
`:figclass:`	CSS class (`margin`)	`:figclass: margin`

Tip

Always add `:alt:` for accessibility. It is the text that screen readers see and displays if the image fails to load.

CHAPTER

EIGHTEEN

EQUATIONS AND MATHEMATICS

MyST supports LaTeX math via the `dollarmath` extension, already enabled in this template. Works in **HTML and PDF**.

18.1 Inline math

Use `$...$` within a line of text:

```
Energy is calculated as $E = mc^2$, where $m$ is mass.
```

Result: Energy is calculated as $E = mc^2$, where m is mass.

18.2 Display equation (unnumbered)

Use `$$...$$` on its own line:

```
$$
\int_0^\infty e^{-x^2} \, dx = \frac{\sqrt{\pi}}{2}
$$
```

Result:

$$\int_0^\infty e^{-x^2} \, dx = \frac{\sqrt{\pi}}{2}$$

18.3 Numbered equation with label

Add a label `(label)` = before the equation to reference it:

```
$$
\frac{\partial u}{\partial t} = \alpha \nabla^2 u
$$ (eq-heat)
```

Result:

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u \tag{18.1}$$

18.4 Cross-reference to an equation

Use `{eq}` to cite an equation by its number:

```
Equation {eq}`eq-heat` describes heat diffusion.
```

Result: Equation (18.1) describes heat diffusion.

18.5 Common math symbols

The following table summarizes the main elements of this section.

Table. Common math symbols.

Symbol	LaTeX code	Symbol	LaTeX code
α	`\alpha`	β	`\beta`
γ	`\gamma`	δ	`\delta`
π	`\pi`	θ	`\theta`
λ	`\lambda`	σ	`\sigma`
$\sum$	`\sum`	$\prod$	`\prod`
$\int$	`\int`	$\oint$	`\oint`
∞	`\infty`	∇	`\nabla`
$\frac{a}{b}$	`\frac{a}{b}`	$\sqrt{x}$	`\sqrt{x}`
$\leq$	`\leq`	$\geq$	`\geq`
$\approx$	`\approx`	$\neq$	`\neq`
$\vec{v}$	`\vec{v}`	$\hat{x}$	`\hat{x}`

Tip

For complex math, ask your AI assistant to generate LaTeX code from a natural language description: “generate the Gauss integral formula in LaTeX.”

CHAPTER

NINETEEN

CITATIONS AND BIBLIOGRAPHY

This template includes `sphinxcontrib-bibtex`, which manages bibliographic references using BibTeX files.

19.1 References file

References are stored in `book/_static/references.bib`. Each entry follows this format:

```
@book{einstein1920,
  author    = {Albert Einstein},
  title     = {Relativity: The Special and General Theory},
  year      = {1920},
  publisher = {Henry Holt and Company}
}

@article{dirac1930,
  author  = {Paul A. M. Dirac},
  title   = {A Theory of Electrons and Protons},
  journal = {Proceedings of the Royal Society A},
  year    = {1930},
  volume  = {126},
  pages   = {360--365}
}
```

19.2 Textual citation: `{cite:t}`

Inserts the author name followed by the year:

```
{cite:t}`einstein1920` formulated the theory of relativity.
```

Result: Einstein [Ein20] formulated the theory of relativity.

19.3 Parenthetical citation: `{cite:p}`

Inserts author and year in parentheses:

```
Quantum mechanics revolutionized physics {cite:p}`dirac1930`.
```

Result: Quantum mechanics revolutionized physics [Dir30].

19.4 Multiple citations

Separate multiple keys with commas:

```
Several authors contributed {cite:p}`einstein1920,dirac1930`.
```

Result: Several authors contributed [Dir30, Ein20].

19.5 Printing the bibliography

Add the `{bibliography}` directive at the end of your page or chapter:

```
```{bibliography}
:filter: docname in docnames
```
```

Result:

19.6 Workflow

1. Add BibTeX entries to `_static/references.bib`.
2. Cite with `{cite:t}` or `{cite:p}` on any page.
3. If you want a local bibliography, add `{bibliography}` at the end of that page.
4. If you want a global bibliography for the whole book, use the `References` page included at the end of the table of contents.

Warning

Each citation key (e.g., `einstein1920`) must be unique across the entire `.bib` file. Duplicates will cause a build failure.

CHAPTER

TWENTY

CROSS-REFERENCES

Cross-references allow you to link figures, tables, equations, and sections within the book. This is **essential** for long documents: if you rearrange chapters, links update automatically.

20.1 Label a section

Write `(label)=` just before the heading:

```
(my-topic)=
## My Important Topic

Content here...
```

20.2 Reference a section

Use `{ref}` to create a link with the heading text:

```
See the {ref}`my-topic` section for details.
```

Result: See the *Figures and Images* section for details.

20.3 Label and reference figures

Use `:name:` inside the `{figure}` directive and `{numref}` to cite by number:

```
```{figure} _static/logo.png
:width: 40%
:name:  to cite by number:



Project logo.
```

The {numref}`fig-logo-ref-en` shows the logo.
```

Result:

The Fig. 20.1 visually summarizes this part of the explanation.

Fig. 20.1: Project logo.

The Fig. 20.1 shows the logo.

20.4 Label and reference equations

Use `(eq-label)=` after the equation and `{eq}` to cite:

```
$$
F = ma
$$ (eq-newton-en)

Equation {eq}`eq-newton-en` is Newton's second law.
```

Result:

$$F = ma \tag{20.1}$$

Equation (20.1) is Newton's second law.

20.5 Label and reference tables

Use `:name:` inside `{table}` and `{numref}` to cite:

```
```{table} Physical constants
:name: tab-constants-en
:align: center

| Constant | Symbol | Value |
|----------|--------|-------|
| Speed of light | $c$ | $3 \times 10^8$ m/s |
| Planck   | $h$     | $6.63 \times 10^{-34}$ J·s |
```

The {numref}`tab-constants-en` lists fundamental constants.
```

Result:

Table 20.1: Physical constants

Constant	Symbol	Value
Speed of light	c	3×10^8 m/s
Planck	h	6.63×10^{-34} J·s

The Table 20.1 lists fundamental constants.

20.6 Roles summary

The following table summarizes the main elements of this section.

Table. Roles summary.

Role	What it references	Example
`{ref}`	Sections (uses heading text)	`{ref}`my-topic`
`{numref}`	Figures and tables (uses number)	`{numref}`fig-logo-ref-en`
`{eq}`	Numbered equations	`{eq}`eq-newton-en`

Tip

Use descriptive label names: `fig-chromatogram`, `eq-bernoulli`, `tab-raw-data`. They are easier to remember than `fig1` or `eq2`.

CHAPTER

TWENTYONE

TABLES

Tables are the most direct way to present structured data. MyST supports standard Markdown tables and the `{table}` directive for adding numbering.

21.1 Simple Markdown table

```
The following table summarizes the main elements of this section.

**Table. Simple Markdown table.**

| Element | Symbol | Atomic number |
|---------|--------|---------------|
| Hydrogen | H     | 1             |
| Helium   | He    | 2             |
| Lithium  | Li    | 3             |
```

Result:

The following table summarizes the main elements of this section.

Table. Simple Markdown table.

Element	Symbol	Atomic number
Hydrogen	H	1
Helium	He	2
Lithium	Li	3

21.2 Numbered table with caption

Use `{table}` for automatic numbering and cross-referencing:

```
```{table} Common laboratory solvents
:name: tab-solvents
:align: center

| Solvent | Formula | Boiling point (°C) |
|---------|---------|---------------------|
| Water   | H₂O     | 100                 |
| Ethanol | C₂H₅OH  | 78                  |
```

(continues on next page)
```

(continued from previous page)

```
| Acetone | CH₃COCH₃| 56                    |
| Ether   | C₄H₁₀O  | 35                    |
```

Result:

Table 21.1: Common laboratory solvents

Solvent	Formula	Boiling point (°C)
Water	H_2O	100
Ethanol	C_2H_5OH	78
Acetone	CH_3COCH_3	56
Ether	$C_4H_{10}O$	35

21.3 List-table directive

MyST does not support per-column alignment in Markdown tables. Use `{list-table}` for more control:

```{list-table} Experimental data
:header-rows: 1
:name: tab-list-en
:align: center

* - Sample
  - pH
  - Conductivity (mS/cm)
* - A
  - 7.2
  - 1.45
* - B
  - 6.8
  - 2.10
* - C
  - 8.1
  - 0.95
```

Result:

Table 21.2: Experimental data

Sample	pH	Conductivity (mS/cm)
A	7.2	1.45
B	6.8	2.10
C	8.1	0.95

21.4 Tips for readable tables

- **Fewer than 6 columns**: wide tables are hard to read on screen.
- **Align numbers**: makes visual comparison easier.
- **Units in the header**: do not repeat units in each cell.
- **Long tables**: if you exceed 15 rows, consider using a chart instead.

Tip

The Table 21.1 shows an example with units in the header. Notice how numeric columns make comparison easier.

CHAPTER

TWENTYTWO

ADMONITIONS AND DROPDOWNS

Admonitions are **highlighted boxes** that draw the reader's attention. They are perfect for hints, warnings, key definitions, and exercise solutions.

22.1 Note (`{note}`)

```
```{note}
The exam will be held on June 15th in room 3.2.
```
```

```
Result:

```{note}
The exam will be held on June 15th in room 3.2.
```
```

22.2 Warning (`{warning}`)

```
```{warning}
Do not mix hydrochloric acid with bleach. It produces toxic gas.
```
```

```
Result:

```{warning}
Do not mix hydrochloric acid with bleach. It produces toxic gas.
```
```

22.3 Tip (`{tip}`)

```
```{tip}
To memorize the periodic table, use a color association technique.
```
```

```
Result:

```{tip}
To memorize the periodic table, use a color association technique.
```
```

22.4 Important ({important})

```
```{important}
The submission deadline is Friday at 23:59. Late submissions are not accepted.
```
```

```
Result:

```{important}
The submission deadline is Friday at 23:59. Late submissions are not accepted.
```
```

22.5 Caution ({caution})

```
```{caution}
This reagent is corrosive. Wear gloves and safety goggles.
```
```

```
Result:

```{caution}
This reagent is corrosive. Wear gloves and safety goggles.
```
```

22.6 Custom admonition

Use `{admonition}` with your own title:

```
```{admonition} Definition: covalent bond
A covalent bond is the union between two atoms that share one or more pairs of␣
↪electrons.
```
```

```
Result:

```{admonition} Definition: covalent bond
A covalent bond is the union between two atoms that share one or more pairs of␣
↪electrons.
```
```

22.7 Dropdown (collapsible content)

Add `:class: dropdown` to hide content until the reader clicks:

```
```{admonition} Solution to exercise 3
:class: dropdown

The molar mass of water (H₂O) is:

$$M = 2 \times 1.008 + 16.00 = 18.016 \text{ g/mol}$$
```
```

```
Result:

```{admonition} Solution to exercise 3
:class: dropdown

The molar mass of water (H₂O) is:

$$M = 2 \times 1.008 + 16.00 = 18.016 \text{ g/mol}$$
```

22.8 Teaching use cases

The following table summarizes the main elements of this section.

Table. Teaching use cases.

Type	Common use
`{note}`	Complementary information, reminders
`{warning}`	Hazards, common errors
`{tip}`	Study tips, tricks
`{important}`	Deadlines, mandatory requirements
`{caution}`	Lab safety precautions
`dropdown`	Exercise solutions, lengthy steps

Tip

Combine admonitions with math, code, or images inside the block. All MyST content works inside an admonition.
```

CHAPTER

TWENTYTHREE

TABS AND CARDS

The **sphinx-design** extension (already installed) lets you organize content into tabs and cards. Ideal for comparing options, showing code in multiple languages, or presenting alternative steps.

23.1 Simple tabs (`{tabbed}`)

```
```{tabbed} Python
```python
print("Hello, world")
```
```

```{tabbed} R
```r
print("Hello, world")
```
```

```{tabbed} MATLAB
```matlab
disp('Hello, world')
```
```
```

Result:

```
```{tabbed} R
```r
print("Hello, world")
```

```
```{tabbed} MATLAB
```matlab
disp('Hello, world')
```

```
## Grouped tabs (`{tab-set}` / `{tab-item}`)

Tab groups allow synchronization:
```

(continues on next page)

(continued from previous page)

```
`` `md
`` `{tab-set}
`` `{tab-item} Analytical method
Solving the differential equation directly.
```

Numerical method

Approximating the solution step by step with a computer.

Result:

Analytical method

Solving the differential equation directly.

Numerical method

Approximating the solution step by step with a computer.

23.2 Tabs for procedure steps

```
`` `{tab-set}
`` `{tab-item} Step 1: Prepare the sample
Weigh 0.5 g of the solid compound with 0.001 g precision.
`` `

`` `{tab-item} Step 2: Dissolve
Add 50 mL of solvent and stir for 5 minutes.
`` `

`` `{tab-item} Step 3: Measure
Read absorbance at 280 nm using the spectrophotometer.
`` `
`` `
```

Result:

Step 1: Prepare the sample

Weigh 0.5 g of the solid compound with 0.001 g precision.

Step 2: Dissolve

Add 50 mL of solvent and stir for 5 minutes.

Step 3: Measure

Read absorbance at 280 nm using the spectrophotometer.

23.3 Cards (`{card}`)

Cards present information in visual blocks:

```
```{card} Lab 1: Spectroscopy
:footer: Week 3

Determination of protein concentration using the Bradford method.
```
```

Result:

Lab 1: Spectroscopy Determination of protein concentration using the Bradford method.

> **Warning**
>
> Tabs and cards work in **HTML** but **not in PDF**. In PDF, the content of all tabs is shown sequentially.

CHAPTER

TWENTYFOUR

CODE BLOCKS

Code blocks are fundamental in science: they show computational formulas, analysis scripts, and step-by-step procedures.

24.1 Syntax highlighting

Add the language after the backticks:

```
```python
import numpy as np

data = np.array([3.2, 4.1, 3.8, 4.5, 3.9])
mean = np.mean(data)
print(f"Mean: {mean:.2f}")
```
```

Result:

```python
import numpy as np

data = np.array([3.2, 4.1, 3.8, 4.5, 3.9])
mean = np.mean(data)
print(f"Mean: {mean:.2f}")
```

24.2 Line numbers

Use `:linenos:` to show line numbers:

```
```python
:linenos:

def calculate_std(data):
    n = len(data)
    mean = sum(data) / n
    variance = sum((x - mean) ** 2 for x in data) / n
    return variance ** 0.5
```
```

Result:

```
:linenos:

def calculate_std(data):
    n = len(data)
    mean = sum(data) / n
    variance = sum((x - mean) ** 2 for x in data) / n
    return variance ** 0.5
```

24.3 Highlight specific lines

Use `:emphasize-lines:` to highlight specific lines:

```
```python
:emphasize-lines: 3

def read_data(filename):
    with open(filename, 'r') as f:
        lines = f.readlines()  # This line will be highlighted
    return [float(l.strip()) for l in lines]
```
```

Result:

```
:emphasize-lines: 3

def read_data(filename):
    with open(filename, 'r') as f:
        lines = f.readlines()
    return [float(l.strip()) for l in lines]
```

24.4 Other languages

Highlighting works with many languages:

```
# Bash example
curl -O https://example.com/data.csv
head -5 data.csv
```

```
-- SQL example
SELECT name, grade
FROM students
WHERE grade >= 5.0
ORDER BY grade DESC;
```

24.5 Common languages in science

The following table summarizes the main elements of this section.

Table. Common languages in science.

Language	When to use it
`python`	Data analysis, numerical computing
`r`	Statistics, biostatistics
`matlab`	Engineering, signal processing
`sql`	Database queries
`bash`	Task automation
`json` / `yaml`	Configuration files

Tip

This template includes `sphinx-copybutton`: a **Copy** button appears in the corner of each code block. Readers can copy code with one click.

CHAPTER

TWENTYFIVE

VIDEOS

Videos greatly enrich teaching materials. However, videos **cannot be embedded in PDF**, so you must always provide a text alternative.

25.1 YouTube video (HTML + PDF)

ALWAYS use this dual pattern with `{raw} html` and `{raw} latex`:

```
```{raw} html
<iframe width="560" height="315" src="https://www.youtube.com/embed/dQw4w9WgXcQ"␣
↪frameborder="0" allowfullscreen></iframe>
```

```{raw} latex
\begin{center}
\textbf{Video:} \url{https://www.youtube.com/watch?v=dQw4w9WgXcQ}
\end{center}
```
```

Result:

Video: https://www.youtube.com/watch?v=dQw4w9WgXcQ

25.2 How to get a YouTube video ID

1. Open the video in your browser.
2. The URL has this format: `https://www.youtube.com/watch?v=VIDEO_ID`
3. Use `VIDEO_ID` in the embed URL: `https://www.youtube.com/embed/VIDEO_ID`

For example, if the URL is `https://www.youtube.com/watch?v=abc123`, the iframe uses `https://www.youtube.com/embed/abc123`.

25.3 Local video (HTML5)

For videos hosted in `_static/`:

```
```{raw} html
<video width="560" controls>
  <source src="_static/my_video.mp4" type="video/mp4">
  Your browser does not support HTML5 video.
</video>
```

```{raw} latex
\begin{center}
\textbf{Local video:} see the digital version to play it.
\end{center}
```
```

Recommended path for local book videos:

- `book/_static/videos/my_video.mp4`

25.4 Animation generated with Manim Community

If you want to create a mathematical or physics animation by code, use **Manim Community Edition** and save the result as an `.mp4` in `book/_static/videos/`.

Example scene:

```
from manim import *
import numpy as np


class SineWave(Scene):
    def construct(self):
        axes = Axes(
            x_range=[0, 2 * np.pi, np.pi / 2],
            y_range=[-1.5, 1.5, 0.5],
            x_length=8,
            y_length=4,
        )
        curve = axes.plot(lambda x: np.sin(x), color=BLUE)
        label = MathTex("y = \\sin(x)").next_to(axes, UP)

        self.play(Create(axes))
        self.play(Create(curve), Write(label))
        self.wait(1)
```

Render command:

```
manim -pql wave.py SineWave
```

Embed it in the book:

```
```{raw} html
<video width="720" controls>
```
```

(continues on next page)

(continued from previous page)

```
  <source src="_static/videos/sine_wave.mp4" type="video/mp4">
  Your browser does not support HTML5 video.
</video>
```

```{raw} latex
\begin{center}
\textbf{Video: Animated sine wave.} See the HTML version of the book.
\end{center}
```

Tip

In this project use ONLY **Manim Community** (`from manim import *`). Do not use ManimGL.

25.5 Video with description

It is good practice to add context before the video:

```
The following video shows the crystallization process of copper sulfate:

[insert video block here]

Duration: 4:32 | Topic: Crystallization
```

25.6 Recommended complete pattern

```
The following video shows the experimental setup:

```{raw} html
<iframe width="560" height="315" src="https://www.youtube.com/embed/VIDEO_ID"␣
↪frameborder="0" allowfullscreen></iframe>
```

```{raw} latex
\begin{center}
\textbf{Video: Experimental setup}\\
\url{https://www.youtube.com/watch?v=VIDEO_ID}
\end{center}
```

Duration: 5:15 | **Topic:** Chemical kinetics
```

Warning

NEVER use only `{raw} html` without the `{raw} latex` block. The PDF will contain no reference to the video and the reader will lose information.

CHAPTER

TWENTYSIX

DIAGRAMS WITH KROKI (MERMAID)

Kroki converts text into SVG diagrams during the book build. If you use `:type: mermaid`, you can write Mermaid syntax and get a diagram that works in both **HTML** and **PDF**.

26.1 Flowchart (`flowchart`)

```
```{kroki}
:type: mermaid
:align: center

flowchart LR
    A[Sample] --> B[Preparation]
    B --> C[Measurement]
    C --> D[Analysis]
    D --> E[Results]
```
```

Result:

The following diagram visually summarizes this part of the explanation.

Diagram: Flowchart (`flowchart`).

Description: The flow goes left to right: Sample → Preparation → Measurement → Analysis → Results.

26.2 Sequence diagram (`sequenceDiagram`)

```
```{kroki}
:type: mermaid
:align: center

sequenceDiagram
    participant S as Student
    participant T as Teacher
    participant L as Laboratory
    S->>T: Submit report
```
```

(continues on next page)

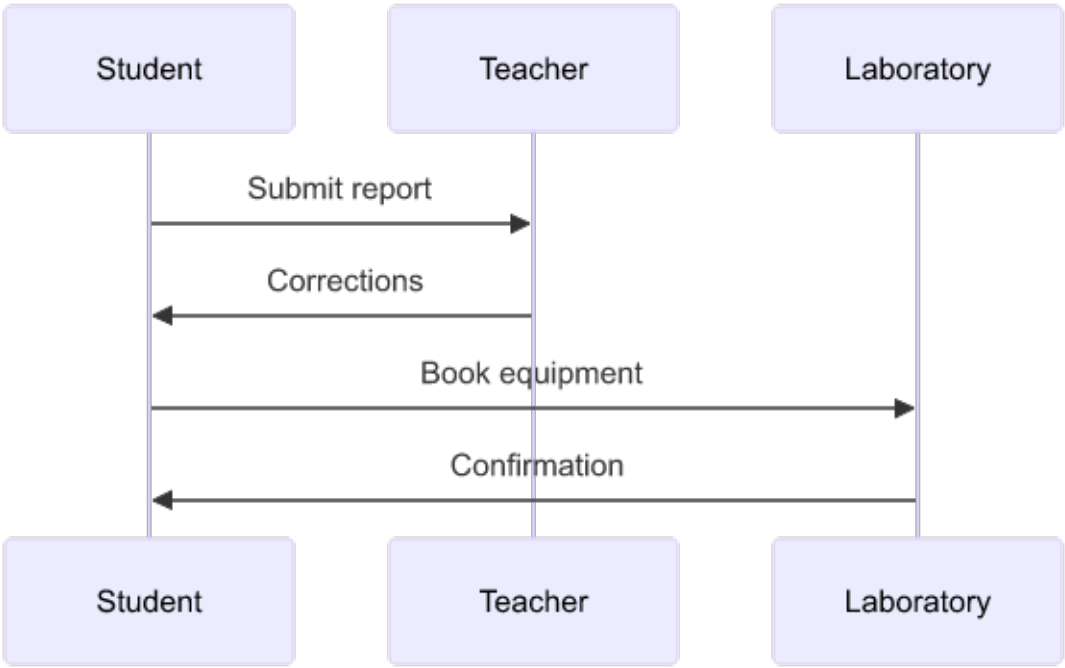
(continued from previous page)

```
    T->>S: Corrections
    S->>L: Book equipment
    L->>S: Confirmation
```

Result:

The following diagram visually summarizes this part of the explanation.

Diagram: Sequence diagram (`sequenceDiagram`).



Description: The student submits a report to the teacher, receives corrections, and then books equipment in the laboratory.

26.3 Class diagram (`classDiagram`)

```{kroki}
:type: mermaid
:align: center

classDiagram
    class Experiment {
        +string name
        +date date
        +run()
        +analyze()
    }
    class Sample {
        +string type
        +float mass
        +prepare()
```

(continues on next page)

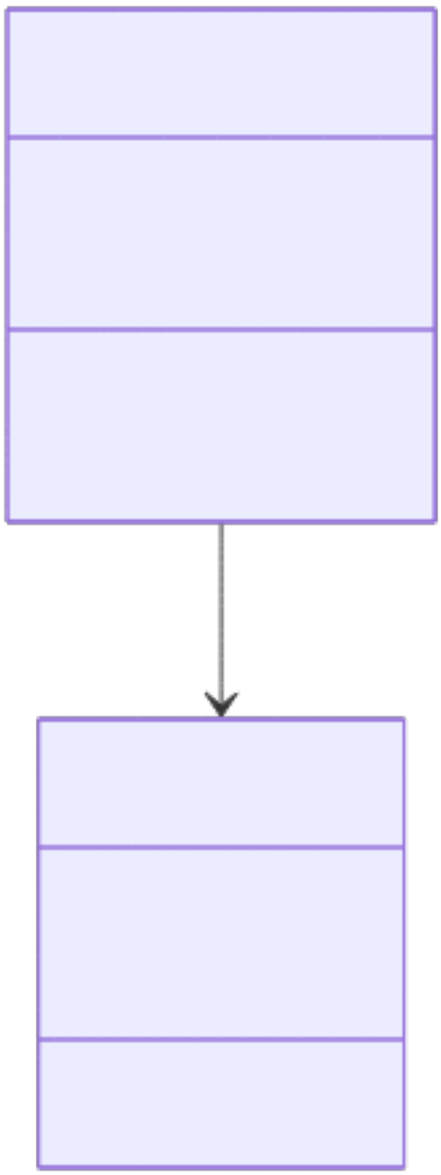
(continued from previous page)

```
    }
    Experiment --> Sample : uses
```

Result:

The following diagram visually summarizes this part of the explanation.

Diagram: Class diagram (`classDiagram`).



26.4 Entity-relationship diagram (`erDiagram`)

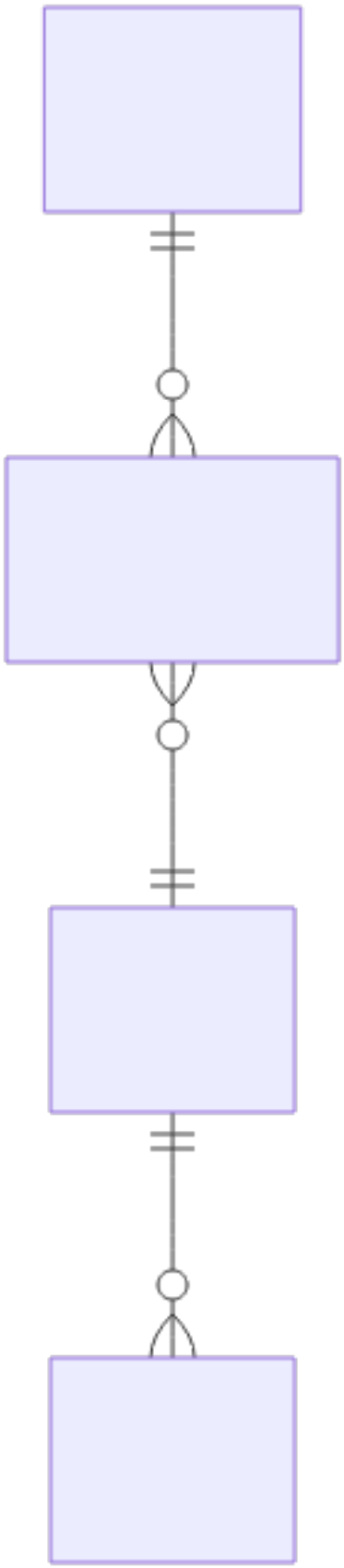
```
```{kroki}
:type: mermaid
:align: center

erDiagram
    STUDENT ||--o{ ENROLLMENT : makes
    ENROLLMENT }o--|| COURSE : belongs_to
    COURSE ||--o{ LAB : includes
```
```

Result:

The following diagram visually summarizes this part of the explanation.

Diagram: Entity-relationship diagram (`erDiagram`).



26.5 State diagram (`stateDiagram-v2`)

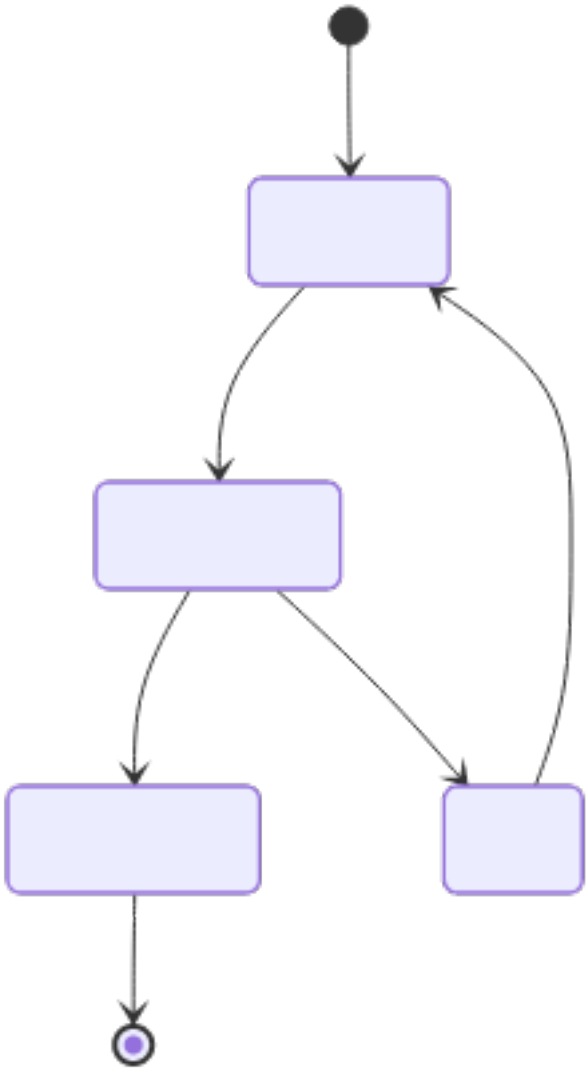
```
```{kroki}
:type: mermaid
:align: center

stateDiagram-v2
    [*] --> Pending
    Pending --> InProgress : start
    InProgress --> Completed : finish
    InProgress --> Error : failure
    Error --> Pending : retry
    Completed --> [*]
```
```

Result:

The following diagram visually summarizes this part of the explanation.

Diagram: State diagram (`stateDiagram-v2`).



26.6 Flowchart directions

The following table summarizes the main elements of this section.

Table. Flowchart directions.

Code	Direction
`flowchart LR`	Left to right
`flowchart RL`	Right to left
`flowchart TB`	Top to bottom
`flowchart BT`	Bottom to top

Tip

Always use `{kroki}` with `:type: mermaid` instead of `{mermaid}`. This makes the diagram work in both HTML and PDF without manual fallbacks.

CHAPTER

TWENTYSEVEN

HTML AND PDF COMPATIBILITY

Your TeachBook is published in two formats: **HTML** (interactive web) and **PDF** (printable version). Not everything works the same in both. This page helps you write content that always works.

27.1 Compatibility table

The following table summarizes the main elements of this section.

Table. Compatibility table.

Element	HTML	PDF	Strategy
Text, lists, links	✅	✅	Use freely
Images (`{image}`, `{figure}`)	✅	✅	Use freely
LaTeX math (`$...$`, `$$...$$`)	✅	✅	Use freely
Tables	✅	✅	Use freely
Admonitions (`{note}`, `{tip}`...)	✅	✅	Use freely
Dropdowns (`:class: dropdown`)	✅	✅ expanded	In PDF it appears expanded
Syntax-highlighted code	✅	✅	Use freely
BibTeX citations	✅	✅	Use freely
Cross-references	✅	✅	Use freely
Kroki diagrams (Mermaid, PlantUML, etc.)	✅	✅	Use `{kroki}` with `:type: mermaid`
Videos (iframe/YouTube)	✅	❌	Use `{raw} latex` with URL
Tabs (`{tabbed}`, `{tab-set}`)	✅	❌	In PDF: sequential content
Interactive HTML (`<details>`)	✅	❌	Use `{raw} latex` alternative
Thebe (live code)	✅	❌	Code visible as text in PDF

27.2 Universal pattern: HTML + LaTeX fallback

For any content that only works in HTML, use this pattern:

```
```{raw} html
<!-- Interactive content here -->
<div>...</div>
```

```{raw} latex
\begin{center}
Alternative text for the PDF.
```
```

(continues on next page)

(continued from previous page)

```
\end{center}
```

27.3 Example: video with fallback

```
```{raw} html
<iframe width="560" height="315" src="https://www.youtube.com/embed/VIDEO_ID"␣
↪frameborder="0" allowfullscreen></iframe>
```

```{raw} latex
\begin{center}
\textbf{Video:} \url{https://www.youtube.com/watch?v=VIDEO_ID}
\end{center}
```
```

27.4 Example: diagram with Kroki

```
```{kroki}
:type: mermaid
:align: center

flowchart LR
    A --> B --> C
```
```

27.5 Practical rules

1. **If it is text, image, equation, or table**: do not worry, it works in both.
2. **If it is interactive (video, custom HTML)**: ALWAYS add a text alternative.
3. **If it is a diagram**: use Kroki. It works in both HTML and PDF.
4. **If it is a dropdown**: in PDF it appears expanded (no problem).
5. **If it uses tabs**: in PDF everything appears sequentially (verify it still makes sense).

Tip

Before publishing, always build both formats (`build_book.py` for HTML and `export_pdf.py` for PDF) and verify the content is coherent in both.

Part V

Interactivity and Assessment

CHAPTER

TWENTYEIGHT

INTERACTIVITY AND ASSESSMENT

This section covers the different ways to add **interactive content** and **assessment** to your TeachBook.

28.1 What will you find here?

Interactivity in a TeachBook ranges from simple dropdowns to live executable code in the browser. This section covers:

The following table summarizes the main elements of this section.

Table. What will you find here?.

Resource	Type	Works in PDF?
Thebe + Binder	Live code with a remote kernel	No
Interactive notebooks	Code executed at build time	Partial
Widgets (ipywidgets)	Interactive controls	No
Quizzes (jupyterquiz)	Self-correcting questions	No
Parametric questions	Exercises with random values	Partial
Exercises with solutions	Dropdowns with answers	Yes
Interactive HTML	Pure JS elements (no frameworks)	No
H5P (optional)	External interactive content	No

Tip

Don't try to use everything at once. Start with **exercises with solutions** (simple and PDF-compatible) and add interactivity as you need it.

Warning

Features marked as experimental may change in future versions. Always verify that interactive content works correctly in your environment before publishing.

28.2 Thebe + Binder: Live Code

Thebe allows readers to run Python code from the web version of the book. In this project it connects to a **remote Binder kernel**.

> **Warning**
>
> Thebe is an **HTML-only** feature. It is not available in PDF export.

28.2.1 How does it work?

1. Click the **"Live Code"** button below.
2. Binder starts a remote environment (this may take a few seconds the first time).
3. Code cells become editable and executable — try it!

28.2.2 Try it: executable code

Click the button to activate interactive mode. Then you can edit and run the cells.

```
import numpy as np

# Create an array from 0 to 2π
x = np.linspace(0, 2 * np.pi, 100)

# Calculate sine and cosine
y_sin = np.sin(x)
y_cos = np.cos(x)

print(f"Mean of sine: {np.mean(y_sin):.4f}")
print(f"Mean of cosine: {np.mean(y_cos):.4f}")
print(f"Max value of sine: {np.max(y_sin):.4f}")
print("It works! Try changing np.sin to another function.")
```

```
import matplotlib.pyplot as plt
import numpy as np

# Modifiable parameters
frequency = 2     # Hz
amplitude = 1.0   # Scale
phase = 0         # Radians

t = np.linspace(0, 2*np.pi, 200)
signal = amplitude * np.sin(frequency * t + phase)

plt.figure(figsize=(8, 3))
plt.plot(t, signal, 'b-', linewidth=2, label=f'{amplitude}·sin({frequency}t + {phase})
↪')
plt.title('Interactive Sine Wave')
plt.xlabel('Time (rad)')
```

(continues on next page)

(continued from previous page)

```
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

28.2.3 How to add this to your pages

1. Activation button

Add this directive where you want the "Live Code" button to appear:

```
```{thebe-button}
```
```

2. Executable cell

Use `code-block` with the `thebe` class:

```
```{code-block} python
:class: thebe

import numpy as np
print("Hello from the browser!")
```
```

3. `_config.yml` configuration

Your `_config.yml` must include the `sphinx-thebe` extension:

```
sphinx:
  extra_extensions:
    - sphinx_thebe
  config:
    thebe_config:
      always_load: false
      repository_url: https://github.com/binder-examples/jupyter-stacks-datascience
      repository_branch: master
```

This configuration uses **Thebe + Binder**, meaning the code runs in a remote Binder environment.

> **Simpler alternative**
>
> If you don't want to set up Thebe, you can use **Google Colab**. Upload your notebook to Colab and share the link in your TeachBook. Students can run the code in the cloud with no installation.

28.3 Interactive Notebooks

This notebook demonstrates how to create interactive content that students can modify. Change the variable values and re-run the cells to see the results.

```
import numpy as np
import matplotlib.pyplot as plt

amplitude = 2.0
frequency = 1.5
phase = np.pi / 4

t = np.linspace(0, 4 * np.pi, 500)
y = amplitude * np.sin(frequency * t + phase)

plt.figure(figsize=(8, 4))
plt.plot(t, y, color='#2563eb', linewidth=2)
plt.title(f'Wave: {amplitude} · sin({frequency}·t + {phase:.2f})')
plt.xlabel('t')
plt.ylabel('y')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

28.3.1 Wave superposition

Modify the frequencies of the two waves to see phenomena like constructive and destructive interference.

```
freq1 = 3.0
freq2 = 3.2

t = np.linspace(0, 10, 1000)
wave1 = np.sin(2 * np.pi * freq1 * t)
wave2 = np.sin(2 * np.pi * freq2 * t)

fig, axes = plt.subplots(3, 1, figsize=(8, 8), sharex=True)

axes[0].plot(t, wave1, color='#2563eb')
axes[0].set_ylabel('Wave 1')
axes[0].set_title(f'f₁ = {freq1} Hz')

axes[1].plot(t, wave2, color='#dc2626')
axes[1].set_ylabel('Wave 2')
axes[1].set_title(f'f₂ = {freq2} Hz')

axes[2].plot(t, wave1 + wave2, color='#16a34a')
axes[2].set_ylabel('Sum')
axes[2].set_title('Wave 1 + Wave 2 (beats)')
axes[2].set_xlabel('Time (s)')

for ax in axes:
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

28.3.2 Exercise

1. Set `freq1 = freq2 = 5.0`. What do you observe?
2. Set `freq1 = 10.0` and `freq2 = 10.5`. What is the beat frequency?
3. Change the amplitude of the second wave and observe the result.

28.4 Interactive Widgets (ipywidgets)

Widgets let you add interactive controls such as sliders, dropdowns, and buttons. They are very useful for letting students explore parameters visually.

```
import numpy as np
import matplotlib.pyplot as plt
from ipywidgets import interact, FloatSlider, Dropdown

@interact(freq=FloatSlider(min=0.5, max=10.0, step=0.1, value=2.0, description=
↪'Frequency'),
         amp=FloatSlider(min=0.1, max=5.0, step=0.1, value=1.0, description=
↪'Amplitude'))
def plot_wave(freq, amp):
    t = np.linspace(0, 2 * np.pi, 500)
    y = amp * np.sin(freq * t)
    plt.figure(figsize=(8, 4))
    plt.plot(t, y, color='#2563eb', linewidth=2)
    plt.ylim(-5.5, 5.5)
    plt.title(f'{amp:.1f} · sin({freq:.1f} · t)')
    plt.xlabel('t')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
```

28.4.1 Function selector

Use the dropdown to choose which function to plot.

```
@interact(func=Dropdown(options=['sin', 'cos', 'tan', 'exp', 'log'], value='sin',␣
↪description='Function'))
def plot_func(func):
    t = np.linspace(-np.pi, np.pi, 500)
    funcs = {'sin': np.sin, 'cos': np.cos, 'tan': np.tan, 'exp': np.exp, 'log': np.
↪log}
    y = funcs[func](t)
    y_clipped = np.clip(y, -5, 5)
    plt.figure(figsize=(8, 4))
    plt.plot(t, y_clipped, color='#16a34a', linewidth=2)
    plt.title(f'f(t) = {func}(t)')
    plt.xlabel('t')
    plt.ylim(-5.5, 5.5)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
```

28.5 Quizzes with jupyterquiz

jupyterquiz lets you embed multiple-choice and numeric questions directly in a notebook. Questions are automatically graded upon answering.

Requirement: `pip install jupyterquiz`

```
from jupyterquiz import display_quiz

physics_questions = [
    {
        "question": "What is the SI unit for force?",
        "type": "multiple_choice",
        "answers": [
            {"answer": "Joule (J)", "correct": False, "feedback": "The joule is a␣
↪unit of energy."},
            {"answer": "Newton (N)", "correct": True, "feedback": "Correct. 1 N = 1␣
↪kg·m/s².”},
            {"answer": "Pascal (Pa)", "correct": False, "feedback": "The pascal is a␣
↪unit of pressure."},
            {"answer": "Watt (W)", "correct": False, "feedback": "The watt is a unit␣
↪of power."}
        ]
    },
    {
        "question": "What is the approximate acceleration due to gravity at Earth's␣
↪surface?",
        "type": "numeric",
        "answers": [
            {"type": "value", "value": 9.8, "correct": True, "feedback": "Correct. g␣
↪≈ 9.8 m/s².”},
            {"type": "range", "range": [9.78, 9.83], "correct": True, "feedback":
↪"Correct, within the expected range."},
            {"type": "value", "value": 10, "correct": False, "feedback": "A common␣
↪approximation, but not the standard value."}
        ]
    },
    {
        "question": "Newton's Second Law is expressed as:",
        "type": "multiple_choice",
        "answers": [
            {"answer": "F = ma", "correct": True, "feedback": "Correct. Force equals␣
↪mass times acceleration."},
            {"answer": "F = mv", "correct": False, "feedback": "mv is momentum."},
            {"answer": "F = mg", "correct": False, "feedback": "This is the weight␣
↪formula, a specific case."},
            {"answer": "E = mc²", "correct": False, "feedback": "This is Einstein's␣
↪mass-energy equivalence."}
        ]
    }
]

display_quiz(physics_questions)
```

28.6 Parametric Questions

Parametric questions generate **random values** on each execution, so each student gets a different problem with the same structure. Re-run the cells to get new values.

```
import random
import math

a = random.randint(1, 10)
b = random.randint(1, 10)
c = random.randint(-5, 5)

print(f"Solve the quadratic equation: {a}x² + {b}x + {c} = 0")
print(f"\nCalculate the discriminant Δ = b² - 4ac and the solutions.")

discriminant = b**2 - 4*a*c
print(f"\n--- Solution (expand to reveal) ---")
print(f"Δ = {b}² - 4·{a}·({c}) = {discriminant}")
if discriminant > 0:
    x1 = (-b + math.sqrt(discriminant)) / (2*a)
    x2 = (-b - math.sqrt(discriminant)) / (2*a)
    print(f"x₁ = {x1:.4f}")
    print(f"x₂ = {x2:.4f}")
elif discriminant == 0:
    x = -b / (2*a)
    print(f"Double root: x = {x:.4f}")
else:
    real = -b / (2*a)
    imag = math.sqrt(-discriminant) / (2*a)
    print(f"x₁ = {real:.4f} + {imag:.4f}i")
    print(f"x₂ = {real:.4f} - {imag:.4f}i")
```

28.6.1 Parallel resistors

Calculate the equivalent resistance and total current.

```
R1 = random.choice([10, 22, 33, 47, 68, 100, 150, 220, 330, 470])
R2 = random.choice([10, 22, 33, 47, 68, 100, 150, 220, 330, 470])
R3 = random.choice([10, 22, 33, 47, 68, 100, 150, 220, 330, 470])
V = random.choice([5, 9, 12, 24])

print(f"Three resistors in parallel: R₁ = {R1} Ω, R₂ = {R2} Ω, R₃ = {R3} Ω")
print(f"Applied voltage: V = {V} V")
print(f"\nCalculate the equivalent resistance Req and total current I.")

Req = 1 / (1/R1 + 1/R2 + 1/R3)
I_total = V / Req

print(f"\n--- Solution ---")
print(f"1/Req = 1/{R1} + 1/{R2} + 1/{R3}")
print(f"Req = {Req:.2f} Ω")
print(f"I = V/Req = {V}/{Req:.2f} = {I_total:.4f} A = {I_total*1000:.2f} mA")
```

28.7 Exercises with Solutions

Patterns for creating exercises with hidden solutions that students can reveal when ready. This approach works in both **HTML** and **PDF**.

28.7.1 Basic pattern

Use `{admonition}` for the exercise statement and `{admonition}` with `:class: dropdown` for the solution:

```
```{admonition} Exercise 1 — Basic derivative
:class: important

Calculate the derivative of $f(x) = 3x^4 - 2x^2 + 7x - 1$.
```

```{admonition} Solution
:class: dropdown

Apply differentiation rules term by term:

$$f'(x) = 12x^3 - 4x + 7$$
```
```

Exercise 1 — Basic derivative

Calculate the derivative of $f(x) = 3x^4 - 2x^2 + 7x - 1$.

Solution

Apply differentiation rules term by term:

$$f'(x) = 12x^3 - 4x + 7$$

28.7.2 Exercises with difficulty levels

Add difficulty hints to guide the student:

Exercise 2 — Integral (level: basic)

Calculate $\int_0^3 (2x + 1)\, dx$.

Solution

$$\int_0^3 (2x+1)\,dx = \left[x^2+x\right]_0^3 = (9+3)-0 = 12$$

Exercise 3 — Integral (level: intermediate)

Calculate $\int_0^\pi x\sin(x)\,dx$. *Hint: integration by parts.*

Solution

Use integration by parts with $u = x$ and $dv = \sin(x)\,dx$:

$$\int_0^\pi x\sin(x)\,dx = \left[-x\cos(x)\right]_0^\pi + \int_0^\pi \cos(x)\,dx = \pi + 0 = \pi$$

Exercise 4 — Integral (level: advanced)

Calculate $\int_{-\infty}^{\infty} e^{-x^2}\,dx$. *Hint: use polar coordinates.*

Solution

Let $I = \int_{-\infty}^{\infty} e^{-x^2}dx$. Then:

$$I^2 = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} e^{-(x^2+y^2)}dx\,dy$$

Converting to polar: $I^2 = \int_0^{2\pi}\int_0^\infty e^{-r^2}r\,dr\,d\theta = 2\pi\cdot\frac{1}{2} = \pi$

Therefore: $I = \sqrt{\pi}$

Tip

Dropdowns appear expanded in PDF export, which is desirable for the printed version. On the web, the student must click to reveal the solution.

28.8 Interactive HTML (no frameworks)

You can create interactive elements using **plain HTML and JavaScript**, without React, Vue, or any framework. All code is self-contained on the page itself.

Warning

Content in `{raw} html` blocks does **not appear in PDF**. Always add a `{raw} latex` block with alternative text for the printed version.

28.8.1 Collapsible content with `<details>`

Hint: Remember that the derivative of e^x is e^x.

28.8.2 Slider with live output

Temperature converter: $T_{^\circ\mathrm{F}} = T_{^\circ\mathrm{C}} \times \frac{9}{5} + 32$

28.8.3 Simple calculator

Ohm's Law calculator: $V = I \times R$

Rules for interactive HTML

1. **No JS frameworks**: do not use React, Vue, Angular, or npm.
2. **No external files**: all JS goes inline inside the `{raw} html` block.
3. **Always include LaTeX fallback**: add a `{raw} latex` block with essential content for PDF.
4. **Inline styles**: use `style=""` on tags to avoid depending on external CSS.

28.9 H5P (Optional)

H5P is an interactive content platform that offers quizzes, drag-and-drop activities, video interactions, and much more.

Warning

H5P is **optional** and **not a dependency** of this template. It requires an external server to host the content (H5P.com, Moodle, WordPress with H5P plugin, etc.). It is not self-contained.

28.9.1 What does H5P offer?

The following table summarizes the main elements of this section.

Table. What does H5P offer?.

Activity type	Description
Quizzes	Multiple choice, fill in the blanks
Drag & Drop	Drag elements to correct zones
Interactive video	Add pauses with questions on a video
Interactive presentation	Slides with integrated questions
Summary	Text with interspersed questions

28.9.2 How to embed H5P in your TeachBook

Create your content on the H5P platform and use an `iframe` to display it:

```
```{raw} html
<iframe src="https://H5P_URL_HERE" width="800" height="600" frameborder="0"␣
↪allowfullscreen="allowfullscreen" style="border: 1px solid #e5e7eb; border-radius:␣
↪8px;"></iframe>
```

```{raw} latex
\textbf{Interactive H5P activity:} Available in the web version of the book.
Link: \url{https://H5P_URL_HERE}
```
```

28.9.3 Compatible platforms

- **H5P.com**: Official service (paid for advanced use).
- **Moodle**: Free plugin, ideal if your university already uses Moodle.
- **WordPress**: Free H5P plugin.
- **Lumi**: Desktop application to create H5P without a server.

Recommended alternative

If you don't need the complexity of H5P, **exercises with solutions** (using `{admonition}` with `dropdown`) cover most assessment needs and work in both HTML and PDF, with no external server required.

Part VI

Information

CHAPTER

TWENTYNINE

ABOUT

You can cite this book as:

> Álvaro Lozano Murciego and André Sales Mendes (2025). *TeachBook (Plantilla Ciencias Químicas)*. https://elloza.com/teachbook_sciences_template/ . Source files at https://github.com/elloza/teachbook_sciences_template . CC BY 4.0.

You can also cite individual pages of the book as:

> `<Page title>`. In Álvaro Lozano Murciego and André Sales Mendes (2025). *TeachBook (Plantilla Ciencias Químicas)*. https://elloza.com/teachbook_sciences_template/ . Source files at https://github.com/elloza/teachbook_sciences_template . CC BY 4.0.

Because this book is expected to evolve over time, it is recommended to cite the source repository as well whenever the exact context of the material matters.

29.1 How this book is made

This website is written in Markdown files and Jupyter notebooks, and converted to HTML using tools from TeachBooks and Jupyter Book.

The source files are stored in a public GitHub repository:

- https://github.com/elloza/teachbook_sciences_template

The published version of the book can be viewed at:

- https://elloza.com/teachbook_sciences_template/

29.2 How to reproduce the website

There are two main options:

1. **On GitHub**: fork the repository, enable GitHub Pages with source `GitHub Actions`, and run the deployment workflow.
2. **Locally**: clone the repository, install dependencies, and build the book with the project scripts.

29.3 About the authors

29.3.1 Authors

- **Álvaro Lozano Murciego**
- **André Sales Mendes**

29.3.2 Institution

Faculty of Sciences, University of Salamanca (USAL)

CHAPTER

THIRTY

LICENSES

30.1 Book license

This book is distributed under the Creative Commons Attribution 4.0 International (CC BY 4.0) license.

This allows the material to be shared and adapted, as long as attribution is provided.

30.2 External resources

Unless explicitly stated on a specific page, the content of this book is authored by the project authors.

If any external resources are reused in a page, they should be acknowledged on that page together with their corresponding attribution and license.

30.3 Code and tools

This project uses external tools and libraries, including:

- TeachBooks
- Jupyter Book
- MyST Markdown

Each tool keeps its own license.

CHAPTER

THIRTYONE

HOW TO CITE

If you use this material, please cite it as follows:

31.1 Text Citation

> Faculty of Sciences (2025). *TeachBook Sciences Template*. University of Salamanca. Available at: [Repository URL]

31.2 BibTeX

```bibtex
@book{teachbook_sciences_2025,
  author = {Faculty of Sciences},
  title = {TeachBook Sciences Template},
  year = {2025},
  publisher = {University of Salamanca},
  url = {https://github.com/user/repo}
}
```

31.3 DOI

(Pending assignment via Zenodo)

CHAPTER

THIRTYTWO

REFERENCES

This page gathers the book bibliography in a single place.

BIBLIOGRAPHY

[Dir30] Paul A. M. Dirac. A theory of electrons and protons. *Proceedings of the Royal Society A*, 126:360–365, 1930.

[Ein20] Albert Einstein. *Relativity: The Special and General Theory*. Henry Holt and Company, 1920.

[Dir30] Paul A. M. Dirac. A theory of electrons and protons. *Proceedings of the Royal Society A*, 126:360–365, 1930.

[Ein20] Albert Einstein. *Relativity: The Special and General Theory*. Henry Holt and Company, 1920.